DATA STRUCTURES AND ALGORITHMS

23CSE203

LAB MANUAL

SUBMITTED BY	
NAME	A. LIKITHA SRI
ROLL NO	AV.SC.U4CSE25124
YEAR/SEM/SECTION	2ND YEAR/SEM-3/CSE-B
CREDITS	05

SUBMITTED TO	DR.RAJ KUMAR BATCHU
DESIGNATION	ASST.PROFESSOR(SI.GRADE)
DEPARTMENT	SCHOOL OF COMPUTING

INDEX

S.NO	TITLE	DATE	PAGE NO
1.	**WEEK 1:** **Recursion** A) Recursive program for countdown from n to 1, followed by 'launch'. B) Recursive program to calculate P^n (compound interest growth factor P, years n). C) Recursive program to print first n terms of Fibonacci series. D) Recursive program to search for an employee ID in a list. E) Recursive program to calculate factorial (n!) for parcel arrangement.	14-07-26	4-13
2.	**WEEK 2:** **Searching** A) Python program to perform Linear Search on given input. B) Python program to perform Binary Search on given input (verify with sorted and unsorted lists).	21-07-26	14-20
3.	**WEEK 3:** **Elementary Sorting** A) Python program to implement Bubble Sort (user input). B) Python program to implement Selection Sort (user input). C) Python program to implement Insertion Sort (user input).	01-08-26	21-26
4.	**WEEK 4:** **Advanced Sorting** A) Python program to implement Merge Sort (user input). B) Python program to implement Quick Sort (user input).	11-08-26	27-31
5.	**WEEK 5:** **Linked Lists** Implement operations for: A) Singly Linked List (SLL) B) Doubly Linked List (DLL) C) Circular Linked List (CLL) Operations: 1. Create LL 2. Insert at Beginning 3. Insert at End 4. Insert at Index 5. Delete by Value 6. Delete	18-08-26	32-69

	Prior Node 7. Delete Last Node 8. Count Nodes 9. Display/Traverse 10. Exit		
6.	**WEEK 6:** **STACKS** A) Implement Stack operations (Push, Pop, Peek, Display) using an Array. B) Implement Stack operations (Push, Pop, Peek, Display) using Linked List.	25-08-26	70-81
7.	**WEEK 7:** **Queues** A) Implement Queue using Arrays. B) Implement Queue using Linked List. C) Implement Circular Queue using Arrays. D) Implement Circular Queue using Linked List.	22-09-26	82-105
8.	**WEEK 8:** **Binary Trees** A) Implement Binary Tree with operations using Array. B) Implement Binary Tree with operations using Linked List.	29-09-26	106-121
9.	**WEEK 9:** **Binary Search Trees** Python program to implement Binary Search Tree (BST) operations.	06-10-26	122-135
10.	**WEEK 10:** **AVL Tree** Write a Python program to implement an AVL Tree with all its operations.	13-10-26	136-147
11.	**WEEK 11:** **Hashing** Python program to implement Hashing (Separate chaining, Linear probing, and Quadratic probing).	27-10-26	148-154

WEEK – 1

PROGRAM-1(A):

AIM: A rocket launch system displays a countdown before launch. Instead of using a loop, implement the countdown using recursion. Write a recursive python program to display a countdown from n to 1, followed by "Launch!".

CODE:

1a.py - C:/Users/amuda/Desktop/DSA25124/1a.py (3.14.0)

File Edit Format Run Options Window Help

```
print("Name: A. Likitha")
print("Roll Number: 25124")
print("--------------------")
def countdown(n):
    if n <= 0:
        print("Launch!")
    else:
        print(n)
        countdown(n - 1)


n = int(input("Enter countdown number: "))

countdown(n)
```

OUTPUT:

```
>>>
================ RESTART: C:/User
Name: A. Likitha
Roll Number: 25124
--------------------
Enter countdown number: 6
6
5
4
3
2
1
Launch!
>>>
```

ERROR TABLE:

S.NO	ERROR	RECTIFICATION
1.	Indentation Error	Use 4 spaces for each level of indentation.
2.	Type Error	Convert the input to an integer using n =int (input ("Enter no. of seconds to launch: ")).

EXPLANATION:

• The countdown(n) function takes an integer n as an argument.

• The if n <= 0 condition checks whether the countdown number has reached zero or a negative value.

• If the condition is true, it prints "Launch!" and stops the recursion.

• Otherwise, the else block prints the current value of n.

• The countdown(n - 1) statement calls the function recursively with the number decreased by 1.

• The input() function gets the countdown number from the user.

• The int() function converts the input into an integer.

• The countdown(n) statement starts the countdown using the number entered by the user.

PROGRAM-1(B):

AIM: Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate P^n(power) where: P= principal growth factor, n= number of years.

CODE:

```
1b.py - C:/Users/amuda/Desktop/DSA25124/1b.py (3.14.0)
File  Edit  Format  Run  Options  Window  Help
print("Name: A. Likitha")
print("ROll Number: 25124")
print("---------------------")
def power(p, n):
    if n == 0:
        return 1
    else:
        return p * power(p, n - 1)


p = float(input("Enter principal growth factor (P): "))
n = int(input("Enter number of years (n): "))

result = power(p, n)

print("P^n =", result)
```

OUTPUT:

```
================ RESTART: C:/Users/amuda/Desktop/DSA25124/1b.py =======
Name: A. Likitha
ROll Number: 25124
---------------------
Enter principal growth factor (P): 5
Enter number of years (n): 2
P^n = 25.0
>>>
```

ERROR TABLE:

S.NO	ERROR	RECTIFICATION
1.	Incorrect output if the recursive call is written as power (p, n + 1) instead of power (p, n - 1).	Change the recursive call to power (p, n - 1) so that n decreases toward the base case.

EXPLANATION:

• The power(p, n) function calculates (p^n) using recursion, where p is the base and n is the exponent.

• The if n == 0 condition checks the base case. If the exponent is zero, the function returns 1.

• The else block multiplies p by the result of power(p, n - 1).

• The power(p, n - 1) statement calls the function recursively, decreasing the exponent by 1 each time.

• The input() function gets the principal growth factor P from the user.

• The int() function converts the number of years n into an integer.

• The result = power(p, n) statement calls the recursive function and stores the calculated power in result.

• The print() statement displays the final result, P^n.

PROGRAM-1(C):

AIM: A HR department stores employee ID's in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list.

CODE:

1c.py - C:/Users/amuda/Desktop/DSA25124/1c.py (3.14.0)

File Edit Format Run Options Window Help

```
print("Name: A.Likitha")
print("Roll Number: 25124")
print("--------------------")
def search_employee(ids, key, index):
    if index == len(ids):
        return -1

    if ids[index] == key:
        return index

    return search_employee(ids, key, index + 1)


ids = list(map(int, input("Enter employee IDs: ").split()))
key = int(input("Enter employee ID to search: "))

result = search_employee(ids, key, 0)

if result != -1:
    print("Employee ID found at position", result + 1)
else:
    print("Employee ID not found")
```

POSITIVE CASE OUTPUT:

```
================ RESTART: C:/Users/amuda/Desktop/DSA25124/1c.py ================
Name: A.Likitha
Roll Number: 25124
--------------------
Enter employee IDs:
================ RESTART: C:/Users/amuda/Desktop/DSA25124/1c.py ================
Name: A.Likitha
Roll Number: 25124
--------------------
Enter employee IDs: 120 121 122 123 124
Enter employee ID to search: 124
Employee ID found at position 5
```

NEGATIVE CASE OUTPUT:

```
================ RESTART: C:/Users/amuda/Desktop/DSA25124/1c.py ================
Name: A.Likitha
Roll Number: 25124
--------------------
Enter employee IDs: 120 121 122 123 124
Enter employee ID to search: 100
Employee ID not found
```

ERROR TABLE:

S.NO	Error	Error Type	Rectification
1.	Missing : after for	Syntax Error	Add :
2.	Wrong Indentation	Indentation Error	Maintain proper indentation

EXPLANATION:

• The search_employee(ids, key, index) function searches for an employee ID in the given list using recursion.

• The ids parameter stores the list of employee IDs, key represents the employee ID to search for, and index indicates the current position.

• The if index == len(ids) condition checks whether the end of the list has been reached.

• If the end of the list is reached without finding the ID, the function returns -1, indicating that the employee ID was not found.

• The if ids[index] == key condition checks whether the current employee ID matches the search key.

• If a match is found, the function returns the current index.

• The search_employee(ids, key, index + 1) statement calls the function recursively to check the next element in the list.

• The input() function accepts employee IDs separated by spaces from the user.

• The map(int, input().split()) statement converts the entered employee IDs into integers and stores them in a list.

• The second input() statement accepts the employee ID to be searched.

• The result = search_employee(ids, key, 0) statement starts the recursive search from index 0.

• If result != -1, the program displays the position of the employee ID by adding 1 to the index.

• Otherwise, the program displays "Employee ID not found".

PROGRAM-1(D):

AIM: In a logistics company, the number of possible ways to arrange n different parcels are calculated using the factorial formula (n!). Develop a recursive program to calculate the factorial of a given number.

CODE:

1d.py - C:/Users/amuda/Desktop/DSA25124/1d.py (3.14.0)

File Edit Format Run Options Window Help

```
print("Name: A.LIkitha")
print("Roll Number: 25124")
print("----------------------")
def factorial(n):
    if n == 0 or n == 1:
        return 1
    else:
        return n * factorial(n - 1)


n = int(input("Enter a number: "))

if n < 0:
    print("Factorial is not defined for negative numbers")
else:
    print("Factorial of", n, "is", factorial(n))
```

POSITIVE CASE OUTPUT:

```
================ RESTART: C:/Users/amuda/Desktop/DSA25124/1d.py ================
Name: A.LIkitha
Roll Number: 25124
----------------------
Enter a number: 5
Factorial of 5 is 120
```

NEGATIVE CASE OUTPUT:

```
================ RESTART: C:/Users/amuda/Desktop/DSA25124/1d.py ================
Name: A.LIkitha
Roll Number: 25124
----------------------
Enter a number: -5
Factorial is not defined for negative numbers
>
```

ERROR TABLE:

S.NO	ERROR	RECTIFICATION
1.	Name Error occurs when a variable or function is used without defining it first.	Define the variable or function before using it in the program.
2.	Value Error occurs when an invalid value, such as a letter, is entered where a number is expected.	Enter a valid numerical value or use proper input validation.

EXPLANATION:

• The factorial(n) function calculates the factorial of a number using recursion.

• The if n == 0 or n == 1 condition checks the base case. If n is 0 or 1, the function returns 1.

• The else block calculates the factorial by multiplying n by the factorial of n - 1.

• The factorial(n - 1) statement calls the function recursively, decreasing the value of n by 1 in each call.

• The input() function accepts a number from the user.

• The int() function converts the input into an integer.

• The if n < 0 condition checks whether the entered number is negative.

• If the number is negative, the program displays a message stating that factorial is not defined for negative integers.

• Otherwise, the program calls factorial(n) and displays the calculated factorial.

PROGRAM-1(E):

AIM: Write a recursive python program to print the first n terms of the Fibonacci series.

CODE:

```
1e.py - C:/Users/amuda/Desktop/DSA25124/1e.py (3.14.0)
File  Edit  Format  Run  Options  Window  Help
print("Name: A.Likitha")
print("Roll number: 25124")
print("--------------------")
def fibonacci(n):
    if n <= 1:
        return n
    else:
        return fibonacci(n - 1) + fibonacci(n - 2)


n = int(input("Enter number of terms: "))

print("Fibonacci series:")

for i in range(n):
    print(fibonacci(i), end=" ")
```

OUTPUT:

```
================ RESTART: C:/Users/amuda/Desktop/DSA25124/1e.py ================
Name: A.Likitha
Roll number: 25124
--------------------
Enter number of terms: 8
Fibonacci series:
0 1 1 2 3 5 8 13
```

ERROR TABLE:

S.NO	ERROR	RECTIFICATION
1.	Value Error if the user enters text instead of an integer.	Enter a valid integer for the number of terms.
2.	Recursion Error if the base cases (n == 0 and n == 1) are not included.	Add both base conditions before the recursive call.

EXPLANATION:

• The fibonacci(n) function generates Fibonacci numbers using recursion.

• The if n <= 1 condition is the base case and returns n when it is 0 or 1.

• The else block calculates the next Fibonacci number by adding the previous two terms: fibonacci(n - 1) + fibonacci(n - 2).

• The input() function accepts the number of terms from the user.

• The for loop calls the fibonacci(i) function for each term and prints the Fibonacci series.

• The end=" " argument prints the numbers on the same line, separated by spaces.

WEEK - 2

PROGRAM-2(A):

AIM: Write a python program to implement linear search.

CODE:

2a.py - C:/Users/amuda/Desktop/DSA25124/2a.py (3.14.0)

File Edit Format Run Options Window Help

```
print("Name: A.Likitha")
print("Roll number: 25124")
print("----------------------")
def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i
    return -1


arr = list(map(int, input("Enter elements: ").split()))
key = int(input("Enter element to search: "))

result = linear_search(arr, key)

if result != -1:
    print("Element found at position", result + 1)
else:
    print("Element not found")
```

POSITIVE CASE OUTPUT:

```
================ RESTART: C:/Users/amuda/Desktop/DSA25124/2a.py ================
Name: A.Likitha
Roll number: 25124
----------------------
Enter elements: 10 20 30 40 50
Enter element to search: 30
Element found at position 3
```

NEGATIVE CASE OUTPUT:

```
================ RESTART: C:/Users/amuda/Desktop/DSA25124/2a.py ============
Name: A.Likitha
Roll number: 25124
----------------------
Enter elements: 10 20 30 40 50
Enter element to search: 35
Element not found
```

ERROR TABLE:

S.NO	ERROR	RECTIFICATION
1.	ValueError occurs if a non-integervalue is entered when int (input ()) is used.	Enter only integer values for the number of elements and array elements.
2.	TypeError may occur if len(arr) is used on a variable that is not a list.	Make sure arr is initialized as a list using arr = [] and elements are appended to it.

EXPLANATION:

• The program performs a linear search to find a specified element in an array.

• The arr[100] array stores the elements, while n, key, and i represent the number of elements, search element, and loop index.

• The scanf() function accepts the number of elements and array values from the user.

• The for loop reads and stores the elements in the array.

• The key variable stores the element to be searched.

• The second for loop compares each array element with the search key.

• If a match is found, the program displays its position, sets found = 1, and stops the loop using break.

• If found remains 0, the program displays "Element not found".

• The return 0 statement indicates successful program execution.

PROGRAM-2(B):

AIM: Write a python program to implement Binary Search with and without sorted array as an input.

CODE:

2b.py - C:/Users/amuda/Desktop/DSA25124/2b.py (3.14.0)

File Edit

```
print("Name: A.Likitha")
print("Roll number:25124")
print("----------------------")
def binary_search(arr, key):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1

    return -1


arr = list(map(int, input("Enter elements: ").split()))
key = int(input("Enter element to search: "))

arr.sort()

print("Sorted array:", arr)

result = binary_search(arr, key)

if result != -1:
    print("Element found at position", result + 1)
else:
    print("Element not found")
```

POSITIVE CASE OUTPUT:

```
================ RESTART: C:/Users/amuda/Desktop/DSA25124/2b.py ================
Name: A.Likitha
Roll number:25124
----------------------
Enter elements: 10 20 30 40 50
Enter element to search: 30
Sorted array: [10, 20, 30, 40, 50]
Element found at position 3
```

NEGATIVE CASE OUTPUT:

```
================ RESTART: C:/Users/amuda/Desktop/DSA25124/2b.py ======
Name: A.Likitha
Roll number:25124
----------------------
Enter elements: 10 20 30 40 50
Enter element to search: 25
Sorted array: [10, 20, 30, 40, 50]
Element not found
```

ERROR TABLE:

S.NO	ERROR	RECTIFICATION
1.	Indentation Error if the statements inside the function or loop are not properly indented.	Maintain proper indentation inside the function and loop.
2.	IndexError if an invalid index is accessed while searching.	Make sure low and high remain within the list range.
3.	ValueError if a non-numeric value is entered instead of an integer.	Enter only valid integer values. valid integer values.

EXPLANATION:

• The program performs a binary search to find a specified element in a sorted array.

• The arr[100] array stores the elements, while n represents the number of elements and key represents the element to be searched.

• The low and high variables represent the starting and ending indices of the search range.

• The scanf() function accepts the number of elements, sorted array values, and search key from the user.

• The mid = (low + high) / 2 statement calculates the middle index of the current search range.

• If arr[mid] == key, the element is found, its position is displayed, and the loop terminates using break.

• If key < arr[mid], the search continues in the left half by updating high = mid - 1.

• If key > arr[mid], the search continues in the right half by updating low = mid + 1.

• The found variable indicates whether the element has been found. If it remains 0, the program displays "Element not found".

• The return 0 statement indicates successful program execution.

AIM: Write a python program to implement Binary Search with and without unsorted array as an input.

CODE:

2b(2).py - C:/Users/amuda/Desktop/DSA25124/2b(2).py (3.14.0)

File Edit Format Run Options Window Help

```
print("Name: A.Likitha")
print("Roll number:25124")
print("--------------------------")
def binary_search(arr, key):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1

    return -1


arr = list(map(int, input("Enter elements: ").split()))
key = int(input("Enter element to search: "))

arr.sort()

print("Sorted array:", arr)

result = binary_search(arr, key)

if result != -1:
    print("Element found at position", result + 1)
else:
    print("Element not found")
```

POSITIVE CASE OUTPUT:

```
=============== RESTART: C:/Users/amuda/Desktop/DSA25124/2b(2).py ======
Name: A.Likitha
Roll number:25124
-------------------------
Enter elements: 24 41 31 10 45
Enter element to search: 41
Sorted array: [10, 24, 31, 41, 45]
Element found at position 4
```

NEGATIVE CASE OUTPUT:

```
=============== RESTART: C:/Users/amuda/Desktop/DSA25124/2b(2).py =======
Name: A.Likitha
Roll number:25124
-------------------------
Enter elements: 24 12 35 42 20
Enter element to search: 55
Sorted array: [12, 20, 24, 35, 42]
Element not found
```

EXPLANATION:

• The program performs a binary search to find a specified element in a sorted array.

• The arr[100] array stores the elements, while n represents the number of elements and key represents the element to be searched.

• The low and high variables represent the starting and ending indices of the search range.

• The scanf() function accepts the number of elements, sorted array values, and search key from the user.

• The mid = (low + high) / 2 statement calculates the middle index of the current search range.

• If arr[mid] == key, the element is found, its position is displayed, and the loop terminates using break.

• If key < arr[mid], the search continues in the left half by updating high = mid - 1.

• If key > arr[mid], the search continues in the right half by updating low = mid + 1.

• The found variable indicates whether the element has been found. If it remains 0, the program displays "Element not found".

• The return 0 statement indicates successful program execution.

WEEK – 3

PROGRAM-3(A):

AIM: Write a python program to implement bubble sort.

CODE:

```
3a.py - C:/Users/amuda/Desktop/DSA25124/3a.py (3.14.0)
File  Edit  Format  Run  Options  Window  Help
def bubble_sort(arr):
    n = len(arr)

    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]

    return arr


arr = list(map(int, input("Enter elements: ").split()))

print("Original array:", arr)

bubble_sort(arr)

print("Sorted array:", arr)
```

OUTPUT:

```
========== RESTART: C:/Users/amuda/Desktop/DSA25124/3a.py
Name: A.Likitha
Roll number:25124
------------------
Enter elements: 15 10 35 45 20
Original array: [15, 10, 35, 45, 20]
Sorted array: [10, 15, 20, 35, 45]
```

ERROR TABLE:

S.NO	ERROR	RECTIFICATION
1.	NameError if arr, n, or another variable is misspelled.	Check the variable names and use the correct spelling.
2.	IndexError may occur if the program accesses an invalid array position.	Use the correct loop range range(n-i-1) to keep the index within the list.

3.	ValueError if a non-integer value is entered as an array element.	Enter only valid integer values and linely.

EXPLANATION:

• The program sorts the elements of an array in ascending order using the Bubble Sort algorithm.

• The arr[100] array stores the elements, while n, i, and j are used to control the loops.

• The scanf() function accepts the number of elements and their values from the user.

• The outer for loop controls the number of passes required to sort the array.

• The inner for loop compares adjacent elements in the array.

• The if (arr[j] > arr[j + 1]) condition checks whether the adjacent elements are in the wrong order.

• The temp variable temporarily stores an element while the two elements are swapped.

• After each pass, the largest unsorted element moves to its correct position at the end of the array.

• The final for loop displays the sorted array in ascending order.

• The return 0 statement indicates successful program execution.

PROGRAM-3(B):

AIM: Write a python program to implement insertion sort.

CODE:

3b.py - C:/Users/amuda/Desktop/DSA25124/3b.py (3.14.0)

File Edit Format Run Options Window Help

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1

        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j = j - 1

        arr[j + 1] = key

    return arr


arr = list(map(int, input("Enter elements: ").split()))

print("Original array:", arr)

insertion_sort(arr)

print("Sorted array:", arr)
```

OUTPUT:

```
================ RESTART: C:/Users/amuda/Desktop/DSA25124/
Name: A.Likitha
Roll number:25124
-------------------------
Enter elements: 24 1 35
Original array: [24, 1, 35]
Sorted array: [1, 24, 35]
```

ERROR TABLE:

S.NO	ERROR	RECTIFICATION
1.	IndentationError if the statements inside the function, for, or while loop are not properly indented.	Maintain proper indentation inside all blocks.
2.	IndexError if an invalid list index is accessed while shifting Elements.	Make sure j >= 0 is checked before accessing arr[j].

3.	ValueError if a non-integer value is entered as an array element.	Enter only valid integer values.

EXPLANATION:

• The program sorts the elements of an array in ascending order using the Insertion Sort algorithm.

• The arr[100] array stores the elements, while n, i, j, and key are used during sorting.

• The scanf() function accepts the number of elements and their values from the user.

• The outer for loop starts from the second element and selects each element to be inserted into its correct position.

• The key = arr[i] statement stores the current element to be inserted into the sorted portion of the array.

• The j = i - 1 statement initializes the index of the element immediately before the key.

• The while loop compares the key with the previous elements and shifts larger elements one position to the right.

• The arr[j + 1] = key statement inserts the key into its correct position.

• The final for loop displays the sorted array in ascending order.

• The return 0 statement indicates successful program execution.

PROGRAM-3(C):

AIM: Write a python program to implement selection sort.

CODE:

3c.py - C:/Users/amuda/Desktop/DSA25124/3c.py (3.14.0)

File Edit Format Run Options Window Help

```
def selection_sort(arr):
    n = len(arr)

    for i in range(n):
        min_index = i

        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j

        arr[i], arr[min_index] = arr[min_index], arr[i]

    return arr


arr = list(map(int, input("Enter elements: ").split()))

print("Original array:", arr)

selection_sort(arr)

print("Sorted array:", arr)
```

OUTPUT:

```
================ RESTART: C:/Users/amuda/Desktop/DSA25124/
Name: A.Likitha
Roll number:25124
--------------------
Enter elements: 25 16 37 55 46
Original array: [25, 16, 37, 55, 46]
Sorted array: [16, 25, 37, 46, 55]
```

ERROR TABLE:

S.NO	ERROR	RECTIFICATION
1.	NameError if variables such as arr, i, j, or min_index are misspelled.	Check the variable names and use the correct spelling.
2.	IndexError if an invalid list index is accessed.	Use the correct loop ranges and ensure the index is within the list range.

3.	ValueError if a non-integer value is entered as an array element.	Enter only valid integer values.

EXPLANATION:

• The program sorts the elements of an array in ascending order using the Selection Sort algorithm.

• The arr[100] array stores the elements, while n, i, j, min, and temp are used during sorting.

• The scanf() function accepts the number of elements and their values from the user.

• The outer for loop selects each position where the smallest remaining element should be placed.

• The min = i statement assumes that the current element is the smallest element in the unsorted portion.

• The inner for loop compares the remaining elements to find the smallest element.

• If a smaller element is found, min = j updates the index of the minimum element.

• The temp variable is used to swap the smallest element with the element at the current position.

• The final for loop displays the sorted array in ascending order.

• The return 0 statement indicates successful program execution.

WEEK-4

PROGRAM -4(A):

AIM: Write a python program to implement quick sort.

CODE:

4a.py - C:/Users/amuda/Desktop/DSA25124/4a.py (3.14.0)

File Edit Format Run Options Window Help

```
def quick_sort(arr):
    if len(arr) <= 1:
        return arr

    pivot = arr[0]

    left = []
    right = []

    for i in range(1, len(arr)):
        if arr[i] < pivot:
            left.append(arr[i])
        else:
            right.append(arr[i])

    return quick_sort(left) + [pivot] + quick_sort(right)


arr = list(map(int, input("Enter elements: ").split()))

print("Original array:", arr)

arr = quick_sort(arr)

print("Sorted array:", arr)
```

OUTPUT:

```
================ RESTART: C:/Users/amuda/Desktop/DSA25124/
Name: A.Likitha
Roll number:25124
-------------------
Enter elements: 25 16 3 46 2
Original array: [25, 16, 3, 46, 2]
Sorted array: [2, 3, 16, 25, 46]
>
```

ERROR TABLE:

S.NO	ERROR	RECTIFICATION
1.	TypeError occurred because	Used integer values of the same suitable data type for sorting.

	incompatible data types were present while comparing the elements.	
2.	Recursion Error occurred because the recursive function did not reach the stopping condition properly.	Added and verified the base condition if len(arr) <= 1.
3.	IndexError occurred when the program tried to access an element outside the valid range of the list.	Corrected the loop conditions and ensured that the indexes remain within the list range.
4.	IndentationError occurred because some statements inside the function and loops were not properly aligned.	Corrected the indentation of the statements according to Python syntax.

EXPLANATION:

- The program sorts the elements of an array in ascending order using the Quick Sort algorithm.
- The quick_sort(arr) function takes a list as input and returns the sorted list.
- The if len(arr) <= 1 condition checks whether the list has zero or one element. If so, it returns the list because it is already sorted.
- pivot = arr[0] selects the first element as the pivot.
- The left list stores elements smaller than the pivot.
- The right list stores elements greater than or equal to the pivot.
- The for loop compares each element with the pivot and places it into the appropriate list.
- quick_sort(left) recursively sorts the left list.
- quick_sort(right) recursively sorts the right list.
- quick_sort(left) + [pivot] + quick_sort(right) combines the sorted left list, pivot, and sorted right list.
- input() accepts the elements from the user, and map(int, ...) converts them into integers.
- The first print() displays the original array, while the second displays the sorted array.

PROGRAM -4(B):

AIM: **Write a python program to implement merge sort.**

CODE:

4b.py - C:/Users/amuda/Desktop/DSA25124/4b.py (3.14.0)

File Edit Format Run Options Window Help

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2

    left = arr[:mid]
    right = arr[mid:]

    left = merge_sort(left)
    right = merge_sort(right)

    return merge(left, right)
def merge(left, right):
    result = []

    i = 0
    j = 0

    while i < len(left) and j < len(right):

        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    while i < len(left):
        result.append(left[i])
        i += 1

    while j < len(right):
        result.append(right[j])
        j += 1

    return result


arr = list(map(int, input("Enter elements: ").split()))

print("Original array:", arr)

arr = merge_sort(arr)

print("Sorted array:", arr)
```

OUTPUT:

```
================ RESTART: C:/Users/amuda/Desktop/DSA25124/
Name: A.Likitha
Roll number:25124
-------------------
Enter elements: 25 42 3 5 67
Original array: [25, 42, 3, 5, 67]
Sorted array: [3, 5, 25, 42, 67]
>
```

ERROR TABLE:

S.NO	ERROR	RECTIFICATION
1.	Index Error occurred when the program tried to access an array position outside its valid range.	Corrected the loop conditions and ensured that the indexes remain within the array boundaries.
2.	NameError occurred because I used an incorrect variable name such as pivot, i, or j.	Checked the variable names and corrected their spelling throughout the program.
3.	ValueError occurred when I entered a non-integer value for an array element.	Entered only valid integer values while providing the input.
4.	Recursion Error occurred when the recursive calls continued without reaching the proper stopping condition.	Ensured that the condition if low < high correctly controls the recursive calls.

EXPLANATION:

- The program sorts the elements of an array in ascending order using the Merge Sort algorithm.
- The merge_sort(arr) function divides the array into smaller parts and sorts them recursively.
- The if len(arr) <= 1 condition checks whether the array has zero or one element. If so, it returns the array because it is already sorted.
- mid = len(arr) // 2 finds the middle index of the array.
- left = arr[:mid] stores the first half of the array.
- right = arr[mid:] stores the second half of the array.
- merge_sort(left) and merge_sort(right) recursively sort both halves.
- The merge(left, right) function combines the two sorted halves into one sorted array.
- result = [] creates an empty list to store the merged elements.
- The variables i and j track the positions in the left and right lists.

- The first while loop compares elements from both lists and adds the smaller element to result.
- The next two while loops add any remaining elements from the left and right lists.
- return result returns the merged, sorted list.
- input() accepts elements from the user, and map(int, ...) converts them into integers.
- The first print() displays the original array, while the second displays the sorted array.

WEEK-5

PROGRAM-5(A):

AIM: Write a python program to implement a Singly Linked List with the below operations.

1) **Create Linked List**
2) **Insert at Beginning**
3) **Insert at End**
4) **Insert at Index**
5) **Delete by Value**
6) **Delete First/Prior Node**
7) **Delete Last Node**
8) **Count number of nodes**
9) **Display or Traverse**
10) **Exit**

CODE:

5a.py - C:/Users/amuda/Desktop/DSA25124/5a.py (3.14.0)

File Edit Format Run Options Window Help

```
# Node class represents one node of the linked list
class Node:
    def __init__(self, data):
        self.data = data        # Store the data
        self.next = None        # Store the address of next node


# Linked List class
class SinglyLinkedList:
    def __init__(self):
        self.head = None        # Initially, the list is empty

    # 1. Create Linked List
    def create(self):
        n = int(input("Enter number of nodes: "))

        for i in range(n):
            data = int(input(f"Enter data for node {i + 1}: "))
            new_node = Node(data)

            # If list is empty, make new node the head
            if self.head is None:
                self.head = new_node
            else:
                # Traverse to the last node
                temp = self.head
                while temp.next is not None:
                    temp = temp.next

                # Link the new node at the end
                temp.next = new_node

        print("Linked List created successfully.")

    # 2. Insert at Beginning
    def insert_beginning(self):
        data = int(input("Enter data: "))

        new_node = Node(data)

        # New node points to the current head
        new_node.next = self.head

        # Make new node the new head
        self.head = new_node
```

5a.py - C:/Users/amuda/Desktop/DSA25124/5a.py (3.14.0)

File Edit Format Run Options Window Help

```
        print("Node inserted at beginning.")


    # 3. Insert at End
    def insert_end(self):
        data = int(input("Enter data: "))

        new_node = Node(data)

        # If list is empty, new node becomes head
        if self.head is None:
            self.head = new_node
        else:
            # Traverse to the last node
            temp = self.head
            while temp.next is not None:
                temp = temp.next

            # Link new node after the last node
            temp.next = new_node

        print("Node inserted at end.")


    # 4. Insert at Index
    def insert_at_index(self):
        index = int(input("Enter index: "))
        data = int(input("Enter data: "))

        # Index cannot be negative
        if index < 0:
            print("Invalid index.")
            return

        new_node = Node(data)

        # Insert at beginning if index is 0
        if index == 0:
            new_node.next = self.head
            self.head = new_node
            print("Node inserted successfully.")
            return

        # Find the node before the required index
        temp = self.head
```

5a.py - C:/Users/amuda/Desktop/DSA25124/5a.py (3.14.0)

File Edit Format Run Options Window Help

```
        temp = self.head

        for i in range(index - 1):
            if temp is None:
                print("Index out of range.")
                return
            temp = temp.next

        # If required position does not exist
        if temp is None:
            print("Index out of range.")
            return

        # Insert the new node
        new_node.next = temp.next
        temp.next = new_node

        print("Node inserted successfully.")


    # 5. Delete by Value
    def delete_by_value(self):
        value = int(input("Enter value to delete: "))

        # If list is empty
        if self.head is None:
            print("Linked List is empty.")
            return

        # If the first node contains the value
        if self.head.data == value:
            self.head = self.head.next
            print("Node deleted successfully.")
            return

        # Search for the value
        temp = self.head

        while temp.next is not None:
            if temp.next.data == value:
                # Skip the node containing the value
                temp.next = temp.next.next
                print("Node deleted successfully.")
                return

            temp = temp.next
```

5a.py - C:/Users/amuda/Desktop/DSA25124/5a.py (3.14.0)

File Edit Format Run Options Window Help

```
            temp = temp.next

        print("Value not found.")


    # 6. Delete First Node
    def delete_first(self):
        # Check if list is empty
        if self.head is None:
            print("Linked List is empty.")
            return

        # Move head to the second node
        self.head = self.head.next

        print("First node deleted successfully.")


    # 7. Delete Last Node
    def delete_last(self):
        # Check if list is empty
        if self.head is None:
            print("Linked List is empty.")
            return

        # If there is only one node
        if self.head.next is None:
            self.head = None
            print("Last node deleted successfully.")
            return

        # Traverse until the second-last node
        temp = self.head

        while temp.next.next is not None:
            temp = temp.next

        # Remove the last node
        temp.next = None

        print("Last node deleted successfully.")
```

5a.py - C:/Users/amuda/Desktop/DSA25124/5a.py (3.14.0)

File Edit Format Run Options Window Help

```
        print("Last node deleted successfully.")


    # 8. Count Number of Nodes
    def count_nodes(self):
        count = 0
        temp = self.head

        # Traverse through the entire list
        while temp is not None:
            count += 1
            temp = temp.next

        print("Number of nodes:", count)


    # 9. Display / Traverse
    def display(self):
        # Check if list is empty
        if self.head is None:
            print("Linked List is empty.")
            return

        temp = self.head

        print("Linked List:", end=" ")

        # Traverse and print every node
        while temp is not None:
            print(temp.data, end=" -> ")
            temp = temp.next

        print("None")


# Create an object of SinglyLinkedList
ll = SinglyLinkedList()
```

5a.py - C:/Users/amuda/Desktop/DSA25124/5a.py (3.14.0)

File Edit Format Run Options Window Help

```
# Create an object of SinglyLinkedList
ll = SinglyLinkedList()


# Menu-driven program
while True:
    print("\n========== SINGLY LINKED LIST ==========")
    print("1. Create Linked List")
    print("2. Insert Beginning")
    print("3. Insert End")
    print("4. Insert at Index")
    print("5. Delete by Value")
    print("6. Delete First Node")
    print("7. Delete Last Node")
    print("8. Count Number of Nodes")
    print("9. Display / Traverse")
    print("10. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        ll.create()

    elif choice == 2:
        ll.insert_beginning()

    elif choice == 3:
        ll.insert_end()

    elif choice == 4:
        ll.insert_at_index()

    elif choice == 5:
        ll.delete_by_value()

    elif choice == 6:
        ll.delete_first()

    elif choice == 7:
        ll.delete_last()

    elif choice == 8:
        ll.count_nodes()

    elif choice == 9:
        ll.display()
```

5a.py - C:/Users/amuda/Desktop/DSA25124/5a.py (3.14.0)

File Edit Format Run Options Window Help

```
    print("1. Create Linked List")
    print("2. Insert Beginning")
    print("3. Insert End")
    print("4. Insert at Index")
    print("5. Delete by Value")
    print("6. Delete First Node")
    print("7. Delete Last Node")
    print("8. Count Number of Nodes")
    print("9. Display / Traverse")
    print("10. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        ll.create()

    elif choice == 2:
        ll.insert_beginning()

    elif choice == 3:
        ll.insert_end()

    elif choice == 4:
        ll.insert_at_index()

    elif choice == 5:
        ll.delete_by_value()

    elif choice == 6:
        ll.delete_first()

    elif choice == 7:
        ll.delete_last()

    elif choice == 8:
        ll.count_nodes()

    elif choice == 9:
        ll.display()

    elif choice == 10:
        print("Program terminated.")
        break

    else:
        print("Invalid choice. Please try again.")
```

OUTPUT:

```
================ RESTART: C:/Users/amuda/Desktop/DSA25124/
Name: A.Likitha
Roll number: 25124
-----------------

========== SINGLY LINKED LIST ==========
1. Create Linked List
2. Insert Beginning
3. Insert End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 1
Enter number of nodes: 5
Enter data for node 1: 14
Enter data for node 2: 25
Enter data for node 3: 33
Enter data for node 4: 48
Enter data for node 5: 55
Linked List created successfully.

========== SINGLY LINKED LIST ==========
1. Create Linked List
2. Insert Beginning
3. Insert End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 2
Enter data: 16
Node inserted at beginning.
```

```
========== SINGLY LINKED LIST ==========
1. Create Linked List
2. Insert Beginning
3. Insert End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 3
Enter data: 48
Node inserted at end.

========== SINGLY LINKED LIST ==========
1. Create Linked List
2. Insert Beginning
3. Insert End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 4
Enter index: 5
Enter data: 23
Node inserted successfully.
```

```
========== SINGLY LINKED LIST ==========
1. Create Linked List
2. Insert Beginning
3. Insert End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 5
Enter value to delete: 33
Node deleted successfully.

========== SINGLY LINKED LIST ==========
1. Create Linked List
2. Insert Beginning
3. Insert End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 6
First node deleted successfully.

========== SINGLY LINKED LIST ==========
1. Create Linked List
2. Insert Beginning
3. Insert End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 7
Last node deleted successfully.
```

```
========== SINGLY LINKED LIST ==========
1. Create Linked List
2. Insert Beginning
3. Insert End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 8
Number of nodes: 5

========== SINGLY LINKED LIST ==========
1. Create Linked List
2. Insert Beginning
3. Insert End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 9
Linked List: 14 -> 25 -> 48 -> 23 -> 55 -> None

========== SINGLY LINKED LIST ==========
1. Create Linked List
2. Insert Beginning
3. Insert End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 10
Program terminated.
```

ERROR TABLE:

S.NO	Error	Error Type	Rectification
1.	self.next = None is written as self.next == None	Syntax Error/Logical Error	Use = for assignment
2.	while temp.next is not None is omitted while finding the last node	Logical Error	Traverse until temp.next becomes None
3.	self.head = new_node is forgotten	Logical Error	Update head after inserting the first node

4.	count += 1 is forgotten	Logical Error	Increment count for every node during traversal

EXPLANATION:

- The program implements a Singly Linked List using Python classes.
- The Node class stores data and a reference to the next node.
- The SinglyLinkedList class manages the linked list using the head pointer.
- The create() function creates a linked list with the number of nodes specified by the user.
- The insert_beginning() function inserts a new node at the beginning.
- The insert_end() function inserts a new node at the end.
- The insert_at_index() function inserts a node at a specified index.
- The delete_by_value() function deletes the first node containing the specified value.
- The delete_first() function removes the first node from the list.
- The delete_last() function removes the last node from the list.
- The count_nodes() function counts and displays the total number of nodes.
- The display() function traverses the list and displays all elements.
- The menu-driven program allows the user to select operations repeatedly.
- The while True loop continues execution until the user selects Exit.
- Invalid choices and invalid indices are handled by displaying appropriate messages.

PROGRAM-5(B):

AIM: Write a python program to implement a Double Linked List with the below operations.

1) **Create Linked List**
2) **Insert at Beginning**
3) **Insert at End**
4) **Insert at Index**
5) **Delete by Value**
6) **Delete First Node**
7) **Delete Last Node**
8) **Count number of nodes**

9) **Display or Traverse**

10) **Exit**

CODE:

5b.py - C:/Users/amuda/Desktop/DSA25124/5b.py (3.14.0)

File Edit Format Run Options Window Help

```
print("Name: A.Likitha")
print("Roll number:25124")
print("--------------------")
# Node class
class Node:
    def __init__(self, data):
        self.data = data        # Store the data
        self.prev = None        # Pointer to previous node
        self.next = None        # Pointer to next node


# Doubly Linked List class
class DoublyLinkedList:
    def __init__(self):
        self.head = None        # Initially the list is empty


    # 1. Create Linked List
    def create(self):
        n = int(input("Enter number of nodes: "))

        for i in range(n):
            data = int(input(f"Enter data for node {i + 1}: "))
            new_node = Node(data)

            # If the list is empty
            if self.head is None:
                self.head = new_node

            else:
                # Traverse to the last node
                temp = self.head

                while temp.next is not None:
                    temp = temp.next

                # Connect the new node
                temp.next = new_node
                new_node.prev = temp

        print("Doubly Linked List created successfully.")
```

5b.py - C:/Users/amuda/Desktop/DSA25124/5b.py (3.14.0)

File Edit Format Run Options Window Help

```
        print("Doubly Linked List created successfully.")


    # 2. Insert at Beginning
    def insert_beginning(self):
        data = int(input("Enter data: "))
        new_node = Node(data)

        # If list is not empty
        if self.head is not None:
            new_node.next = self.head
            self.head.prev = new_node

        # Make new node the head
        self.head = new_node

        print("Node inserted at beginning.")


    # 3. Insert at End
    def insert_end(self):
        data = int(input("Enter data: "))
        new_node = Node(data)

        # If list is empty
        if self.head is None:
            self.head = new_node
            print("Node inserted at end.")
            return

        # Traverse to the last node
        temp = self.head

        while temp.next is not None:
            temp = temp.next

        # Connect the new node
        temp.next = new_node
        new_node.prev = temp

        print("Node inserted at end.")
```

5b.py - C:/Users/amuda/Desktop/DSA25124/5b.py (3.14.0)

File Edit Format Run Options Window Help

```
        print("Node inserted at end.")


    # 4. Insert at Index
    def insert_at_index(self):
        index = int(input("Enter index: "))
        data = int(input("Enter data: "))

        # Index cannot be negative
        if index < 0:
            print("Invalid index.")
            return

        new_node = Node(data)

        # Insert at beginning
        if index == 0:
            new_node.next = self.head

            if self.head is not None:
                self.head.prev = new_node

            self.head = new_node

            print("Node inserted successfully.")
            return

        # Find the node before the required index
        temp = self.head

        for i in range(index - 1):
            if temp is None:
                print("Index out of range.")
                return

            temp = temp.next

        # Check if index is out of range
        if temp is None:
            print("Index out of range.")
            return

        # Connect the new node
        new_node.next = temp.next
        new_node.prev = temp
```

5b.py - C:/Users/amuda/Desktop/DSA25124/5b.py (3.14.0)

File Edit Format Run Options Window Help

```
            return

        # Connect the new node
        new_node.next = temp.next
        new_node.prev = temp

        # If new node is not inserted at the end
        if temp.next is not None:
            temp.next.prev = new_node

        temp.next = new_node

        print("Node inserted successfully.")


    # 5. Delete by Value
    def delete_by_value(self):
        value = int(input("Enter value to delete: "))

        # Check if list is empty
        if self.head is None:
            print("Linked List is empty.")
            return

        temp = self.head

        # Search for the value
        while temp is not None:

            if temp.data == value:

                # If deleting the first node
                if temp.prev is None:
                    self.head = temp.next

                    if self.head is not None:
                        self.head.prev = None

                else:
                    # Connect previous node to next node
                    temp.prev.next = temp.next

                    # Connect next node to previous node
                    if temp.next is not None:
                        temp.next.prev = temp.prev
```

5b.py - C:/Users/amuda/Desktop/DSA25124/5b.py (3.14.0)

File Edit Format Run Options Window Help

```
                    temp.prev.next = temp.next

                    # Connect next node to previous node
                    if temp.next is not None:
                        temp.next.prev = temp.prev

                print("Node deleted successfully.")
                return

            temp = temp.next

        print("Value not found.")


    # 6. Delete First Node
    def delete_first(self):

        # Check if list is empty
        if self.head is None:
            print("Linked List is empty.")
            return

        # Move head to the next node
        self.head = self.head.next

        # Remove previous pointer of new head
        if self.head is not None:
            self.head.prev = None

        print("First node deleted successfully.")


    # 7. Delete Last Node
    def delete_last(self):

        # Check if list is empty
        if self.head is None:
            print("Linked List is empty.")
            return

        # If there is only one node
        if self.head.next is None:
            self.head = None
            print("Last node deleted successfully.")
            return
```

5b.py - C:/Users/amuda/Desktop/DSA25124/5b.py (3.14.0)

File Edit Format Run Options Window Help

```
        # Traverse to the last node
        temp = self.head

        while temp.next is not None:
            temp = temp.next

        # Remove the last node
        temp.prev.next = None

        print("Last node deleted successfully.")


    # 8. Count Number of Nodes
    def count_nodes(self):
        count = 0
        temp = self.head

        # Traverse through the list
        while temp is not None:
            count += 1
            temp = temp.next

        print("Number of nodes:", count)


    # 9. Display / Traverse
    def display(self):

        # Check if list is empty
        if self.head is None:
            print("Linked List is empty.")
            return

        temp = self.head

        print("Doubly Linked List:", end=" ")

        # Traverse from beginning to end
        while temp is not None:
            print(temp.data, end=" <-> ")
            temp = temp.next

        print("None")
```

5b.py - C:/Users/amuda/Desktop/DSA25124/5b.py (3.14.0)

File Edit Format Run Options Window Help

```
# Create object of DoublyLinkedList
dll = DoublyLinkedList()


# Menu-driven program
while True:

    print("\n========== DOUBLY LINKED LIST ==========")
    print("1. Create Linked List")
    print("2. Insert Beginning")
    print("3. Insert End")
    print("4. Insert at Index")
    print("5. Delete by Value")
    print("6. Delete First Node")
    print("7. Delete Last Node")
    print("8. Count Number of Nodes")
    print("9. Display / Traverse")
    print("10. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        dll.create()

    elif choice == 2:
        dll.insert_beginning()

    elif choice == 3:
        dll.insert_end()

    elif choice == 4:
        dll.insert_at_index()

    elif choice == 5:
        dll.delete_by_value()

    elif choice == 6:
        dll.delete_first()

    elif choice == 7:
        dll.delete_last()

    elif choice == 8:
        dll.count_nodes()
```

```
    elif choice == 9:
        dll.display()

    elif choice == 10:
        print("Program terminated.")
        break

    else:
        print("Invalid choice. Please try again.")
```

OUTPUT:

```
================ RESTART: C:/Users/amuda/Desktop/DSA25124/5b.py
Name: A.Likitha
Roll number:25124
-------------------

========== DOUBLY LINKED LIST ==========
1. Create Linked List
2. Insert Beginning
3. Insert End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 1
Enter number of nodes: 6
Enter data for node 1: 25
Enter data for node 2: 48
Enter data for node 3: 31
Enter data for node 4: 10
Enter data for node 5: 2
Enter data for node 6: 14
Doubly Linked List created successfully.

========== DOUBLY LINKED LIST ==========
1. Create Linked List
2. Insert Beginning
3. Insert End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 2
Enter data: 54
Node inserted at beginning.
```

```
1. Create Linked List
2. Insert Beginning
3. Insert End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 3
Enter data: 68
Node inserted at end.

========== DOUBLY LINKED LIST ==========
1. Create Linked List
2. Insert Beginning
3. Insert End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 4
Enter index: 54
Enter data: 22
Index out of range.

========== DOUBLY LINKED LIST ==========
1. Create Linked List
2. Insert Beginning
3. Insert End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 5
Enter value to delete: 31
Node deleted successfully.
```

```
========== DOUBLY LINKED LIST ==========
1. Create Linked List
2. Insert Beginning
3. Insert End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 6
First node deleted successfully.

========== DOUBLY LINKED LIST ==========
1. Create Linked List
2. Insert Beginning
3. Insert End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 7
Last node deleted successfully.

========== DOUBLY LINKED LIST ==========
1. Create Linked List
2. Insert Beginning
3. Insert End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 8
Number of nodes: 5
```

```
========== DOUBLY LINKED LIST ==========
1. Create Linked List
2. Insert Beginning
3. Insert End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 9
Doubly Linked List: 25 <-> 48 <-> 10 <-> 2 <-> 14 <-> None

========== DOUBLY LINKED LIST ==========
1. Create Linked List
2. Insert Beginning
3. Insert End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 10
Program terminated.
```

ERROR TABLE:

S.NO	Error	Error Type	Rectification
1.	Trying to access temp.prev when temp is None	Runtime Error	Check that the node exists before accessing it
2.	prev and next are interchanged	Logical Error	Use prev for the previous node and next for the next node
3.	Only temp.next = None is changed while deleting the last node	Logical Error	Ensure the second-last node becomes the last node

EXPLANATION:

- The program implements a Doubly Linked List using Python classes.
- The Node class stores data and two pointers: prev points to the previous node, and next points to the next node.
- The DoublyLinkedList class manages the list using the head pointer.
- The create() function creates a doubly linked list with the number of nodes specified by the user.

- The insert_beginning() function inserts a new node at the beginning and updates the pointers.
- The insert_end() function inserts a new node at the end of the list.
- The insert_at_index() function inserts a node at a specified index and adjusts both pointers.
- The delete_by_value() function searches for and deletes the first node containing the given value.
- The delete_first() function removes the first node and updates the head pointer.
- The delete_last() function removes the last node from the list.
- The count_nodes() function counts and displays the total number of nodes.
- The display() function traverses the list and displays elements using <-> to show links between nodes.
- The menu-driven program allows the user to select and perform different operations.
- The while True loop repeats the menu until the user selects Exit.
- Invalid choices and indices are handled by displaying appropriate messages.

PROGRAM-5(C):

AIM: Write a python program to implement a Circular Linked List with the below operations.

1) **Create Linked List**
2) **Insert Beginning**
3) **Insert End**
4) **Insert at Index**
5) **Delete by Value**
6) **Delete First Node**
7) **Delete Last Node**
8) **Count number of nodes**
9) **Display or Traverse**
10) **Exit**

CODE:

5c.py - C:/Users/amuda/Desktop/DSA25124/5c.py (3.14.0)

File Edit Format Run Options Window Help

```
print("Name: A.Likitha")
print("Roll number: 25124")
print("------------------------")
# Node class
class Node:
    def __init__(self, data):
        self.data = data        # Store the data
        self.next = None        # Pointer to the next node


# Circular Linked List class
class CircularLinkedList:
    def __init__(self):
        self.head = None        # Initially, the list is empty


    # 1. Create Linked List
    def create(self):
        n = int(input("Enter number of nodes: "))

        for i in range(n):
            data = int(input(f"Enter data for node {i + 1}: "))
            new_node = Node(data)

            # If the list is empty
            if self.head is None:
                self.head = new_node
                new_node.next = self.head

            else:
                # Traverse to the last node
                temp = self.head

                while temp.next != self.head:
                    temp = temp.next

                # Insert new node at the end
                temp.next = new_node
                new_node.next = self.head

        print("Circular Linked List created successfully.")
```

5c.py - C:/Users/amuda/Desktop/DSA25124/5c.py (3.14.0)

File Edit Format Run Options Window Help

```
        print("Circular Linked List created successfully.")


    # 2. Insert at Beginning
    def insert_beginning(self):
        data = int(input("Enter data: "))
        new_node = Node(data)

        # If list is empty
        if self.head is None:
            self.head = new_node
            new_node.next = self.head

        else:
            # Find the last node
            temp = self.head

            while temp.next != self.head:
                temp = temp.next

            # New node points to current head
            new_node.next = self.head

            # Last node points to new head
            temp.next = new_node

            # Update head
            self.head = new_node

        print("Node inserted at beginning.")


    # 3. Insert at End
    def insert_end(self):
        data = int(input("Enter data: "))
        new_node = Node(data)

        # If list is empty
        if self.head is None:
            self.head = new_node
            new_node.next = self.head

        else:
            # Find the last node
            temp = self.head
```

5c.py - C:/Users/amuda/Desktop/DSA25124/5c.py (3.14.0)

File Edit Format Run Options Window Help

```
            while temp.next != self.head:
                temp = temp.next

            # Last node points to new node
            temp.next = new_node

            # New node points to head
            new_node.next = self.head

        print("Node inserted at end.")


    # 4. Insert at Index
    def insert_at_index(self):
        index = int(input("Enter index: "))
        data = int(input("Enter data: "))

        # Index cannot be negative
        if index < 0:
            print("Invalid index.")
            return

        # Insert at beginning
        if index == 0:
            new_node = Node(data)

            # If list is empty
            if self.head is None:
                self.head = new_node
                new_node.next = self.head

            else:
                # Find last node
                temp = self.head

                while temp.next != self.head:
                    temp = temp.next

                new_node.next = self.head
                temp.next = new_node
                self.head = new_node

            print("Node inserted successfully.")
            return
```

5c.py - C:/Users/amuda/Desktop/DSA25124/5c.py (3.14.0)

File Edit Format Run Options Window Help

```
        # If list is empty and index is not 0
        if self.head is None:
            print("Index out of range.")
            return

        # Find the node before the required index
        temp = self.head

        for i in range(index - 1):

            temp = temp.next

            # If we have reached the head again,
            # the index is out of range
            if temp == self.head:
                print("Index out of range.")
                return

        # Create new node
        new_node = Node(data)

        # Insert new node
        new_node.next = temp.next
        temp.next = new_node

        print("Node inserted successfully.")


    # 5. Delete by Value
    def delete_by_value(self):
        value = int(input("Enter value to delete: "))

        # Check if list is empty
        if self.head is None:
            print("Linked List is empty.")
            return

        current = self.head
        previous = None

        # Search for the value
        while True:

            if current.data == value:
```

5c.py - C:/Users/amuda/Desktop/DSA25124/5c.py (3.14.0)

File Edit Format Run Options Window Help

```
                # If deleting the first node
                if current == self.head:

                    # If there is only one node
                    if current.next == self.head:
                        self.head = None

                    else:
                        # Find the last node
                        last = self.head

                        while last.next != self.head:
                            last = last.next

                        # Move head to next node
                        self.head = current.next

                        # Last node points to new head
                        last.next = self.head

                else:
                    # Skip the node to be deleted
                    previous.next = current.next

                print("Node deleted successfully.")
                return

            previous = current
            current = current.next

            # If we reached head again,
            # value was not found
            if current == self.head:
                break

        print("Value not found.")


    # 6. Delete First Node
    def delete_first(self):

        # Check if list is empty
        if self.head is None:
            print("Linked List is empty.")
            return
```

5c.py - C:/Users/amuda/Desktop/DSA25124/5c.py (3.14.0)

File Edit Format Run Options Window Help

```
            return

        # If there is only one node
        if self.head.next == self.head:
            self.head = None

        else:
            # Find the last node
            temp = self.head

            while temp.next != self.head:
                temp = temp.next

            # Move head to the next node
            self.head = self.head.next

            # Last node points to new head
            temp.next = self.head

        print("First node deleted successfully.")


    # 7. Delete Last Node
    def delete_last(self):

        # Check if list is empty
        if self.head is None:
            print("Linked List is empty.")
            return

        # If there is only one node
        if self.head.next == self.head:
            self.head = None
            print("Last node deleted successfully.")
            return

        # Find the last and second-last nodes
        previous = self.head
        current = self.head.next

        while current.next != self.head:
            previous = current
            current = current.next
```

5c.py - C:/Users/amuda/Desktop/DSA25124/5c.py (3.14.0)

File Edit Format Run Options Window Help

```
        while current.next != self.head:
            previous = current
            current = current.next

        # Second-last node points to head
        previous.next = self.head

        print("Last node deleted successfully.")


    # 8. Count Number of Nodes
    def count_nodes(self):
        # If list is empty
        if self.head is None:
            print("Number of nodes: 0")
            return

        count = 0
        temp = self.head

        # Traverse until we reach head again
        while True:
            count += 1
            temp = temp.next

            if temp == self.head:
                break

        print("Number of nodes:", count)


    # 9. Display / Traverse
    def display(self):

        # Check if list is empty
        if self.head is None:
            print("Circular Linked List is empty.")
            return

        temp = self.head

        print("Circular Linked List:", end=" ")

        # Traverse the circular list
        while True:
            print(temp.data, end=" -> ")
            temp = temp.next
```

5c.py - C:/Users/amuda/Desktop/DSA25124/5c.py (3.14.0)

File Edit Format Run Options Window Help

```
                break

        print("(back to head)")


# Create an object of CircularLinkedList
cll = CircularLinkedList()


# Menu-driven program
while True:

    print("\n========== CIRCULAR LINKED LIST ==========")
    print("1. Create Linked List")
    print("2. Insert Beginning")
    print("3. Insert End")
    print("4. Insert at Index")
    print("5. Delete by Value")
    print("6. Delete First Node")
    print("7. Delete Last Node")
    print("8. Count Number of Nodes")
    print("9. Display / Traverse")
    print("10. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        cll.create()

    elif choice == 2:
        cll.insert_beginning()

    elif choice == 3:
        cll.insert_end()

    elif choice == 4:
        cll.insert_at_index()

    elif choice == 5:
        cll.delete_by_value()

    elif choice == 6:
        cll.delete_first()

    elif choice == 7:
        cll.delete_last()
```

```
    elif choice == 8:
        cll.count_nodes()

    elif choice == 9:
        cll.display()

    elif choice == 10:
        print("Program terminated.")
        break

    else:
        print("Invalid choice. Please try again.")
```

OUTPUT:

```
================ RESTART: C:/Users/amuda/Desktop/DSA25124/5c.p
Name: A.Likitha
Roll number: 25124
------------------------

========== CIRCULAR LINKED LIST ==========
1. Create Linked List
2. Insert Beginning
3. Insert End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 1
Enter number of nodes: 4
Enter data for node 1: 4
Enter data for node 2: 12
Enter data for node 3: 35
Enter data for node 4: 64
Circular Linked List created successfully.

========== CIRCULAR LINKED LIST ==========
1. Create Linked List
2. Insert Beginning
3. Insert End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 2
Enter data: 45
Node inserted at beginning.
```

```
========== CIRCULAR LINKED LIST ==========
1. Create Linked List
2. Insert Beginning
3. Insert End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 3
Enter data: 52
Node inserted at end.

========== CIRCULAR LINKED LIST ==========
1. Create Linked List
2. Insert Beginning
3. Insert End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 4
Enter index: 24
Enter data: 12
Index out of range.

========== CIRCULAR LINKED LIST ==========
1. Create Linked List
2. Insert Beginning
3. Insert End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 5
Enter value to delete: 4
Node deleted successfully.
```

```
========== CIRCULAR LINKED LIST ==========
1. Create Linked List
2. Insert Beginning
3. Insert End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 6
First node deleted successfully.

========== CIRCULAR LINKED LIST ==========
1. Create Linked List
2. Insert Beginning
3. Insert End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 7
Last node deleted successfully.

========== CIRCULAR LINKED LIST ==========
1. Create Linked List
2. Insert Beginning
3. Insert End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 8
Number of nodes: 3
```

```
========== CIRCULAR LINKED LIST ==========
1. Create Linked List
2. Insert Beginning
3. Insert End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 9
Circular Linked List: 12 -> 35 -> 64 -> (back to head)

========== CIRCULAR LINKED LIST ==========
1. Create Linked List
2. Insert Beginning
3. Insert End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 10
Program terminated.
```

ERROR TABLE:

S.NO	Error	Error Type	Rectification
1.	Using while temp is not None for traversal	Logical Error	Stop when temp == self.head
2.	Deleting the only node without setting head to None	Logical Error	Set self.head = None when only one node exists
3.	Infinite loop during traversal	Logical Error	Stop traversal when the pointer reaches head again
4.	Writing new_node.next = None when creating the first node	Logical Error	Set new_node.next = self.head

EXPLANATION:

- The program implements a Circular Linked List using Python classes.
- The Node class stores data and a pointer to the next node.
- The CircularLinkedList class manages the list using the head pointer.

- The create() function creates a circular linked list in which the last node points back to the first node.
- The insert_beginning() function inserts a new node at the beginning and updates the head pointer.
- The insert_end() function inserts a new node at the end and links it back to the head.
- The insert_at_index() function inserts a new node at the specified index.
- The delete_by_value() function searches for and deletes the first node containing the given value.
- The delete_first() function deletes the first node and updates the head and last-node links.
- The delete_last() function removes the last node and connects the second-last node to the head.
- The count_nodes() function counts the nodes by traversing the list until it reaches the head again.
- The display() function displays all elements and stops when it reaches the head again.
- The menu-driven program allows the user to select different operations.
- The while True loop repeatedly displays the menu until the user selects Exit.
- Invalid choices and indices are handled by displaying appropriate messages.

WEEK-6

PROGRAM:

AIM: Write a python program to implement stack operations such as push, pop, peek, display using an array and a linked list.

PROGRAM 6A: Write a python program to implement stack operations such as push, pop, peek, display using an array.

CODE:

6a.py - C:/Users/amuda/Desktop/DSA25124/6a.py (3.14.0)

File Edit Format Run Options Window Help

```
print("Name: A.Likitha")
print("Roll number: 25124")
print("----------------------")
# Stack implementation using Array

stack = []
MAX = 5

def push():
    if len(stack) == MAX:
        print("Stack Overflow")
    else:
        value = int(input("Enter value: "))
        stack.append(value)
        print(value, "pushed into stack")

def pop():
    if len(stack) == 0:
        print("Stack Underflow")
    else:
        value = stack.pop()
        print(value, "popped from stack")

def peek():
    if len(stack) == 0:
        print("Stack is empty")
    else:
        print("Top element:", stack[-1])

def display():
    if len(stack) == 0:
        print("Stack is empty")
    else:
        print("Stack elements:")
        for i in range(len(stack) - 1, -1, -1):
            print(stack[i])

while True:
    print("\n--- STACK USING ARRAY ---")
    print("1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))
```

```
    if choice == 1:
        push()
    elif choice == 2:
        pop()
    elif choice == 3:
        peek()
    elif choice == 4:
        display()
    elif choice == 5:
        break
    else:
        print("Invalid choice")
```

OUTPUT:

```
================ RESTART: C:/Users/amu
Name: A.Likitha
Roll number: 25124
----------------------

--- STACK USING ARRAY ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter value: 12
12 pushed into stack

--- STACK USING ARRAY ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter value: 15
15 pushed into stack

--- STACK USING ARRAY ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter value: 18
18 pushed into stack

--- STACK USING ARRAY ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter value: 20
20 pushed into stack
```

```
--- STACK USING ARRAY ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter value: 25
25 pushed into stack

--- STACK USING ARRAY ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 2
25 popped from stack

--- STACK USING ARRAY ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 3
Top element: 20

--- STACK USING ARRAY ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
Stack elements:
20
18
15
12
```

```
--- STACK USING ARRAY ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 2
20 popped from stack

--- STACK USING ARRAY ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 2
18 popped from stack

--- STACK USING ARRAY ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 5
```

ERROR TABLE:

S.NO	ERROR	RECTIFICATION
1.	NameError occurred because stack or a function name was misspelled.	Corrected the variable and function names.
2.	SyntaxError occurred because of incorrect Python syntax.	Checked and corrected the syntax of the program.
3.	ValueError occurred when a non-integer value was entered.	Entered only valid integer values.

EXPLANATION:

- The program implements a Stack using an Array (Python list) following the LIFO (Last In, First Out) principle.
- stack = [] creates an empty stack, and MAX = 5 sets its maximum capacity to five elements.
- The push() function adds an element to the top of the stack. If the stack is full, it displays Stack Overflow.

- The pop() function removes and returns the top element. If the stack is empty, it displays Stack Underflow.
- The peek() function displays the top element without removing it.
- The display() function prints all stack elements from top to bottom.
- len(stack) checks the current number of elements in the stack.
- stack.append(value) inserts an element, while stack.pop() removes the top element.
- The while True loop repeatedly displays the menu and allows the user to perform stack operations.
- The if-elif-else statements execute the selected operation or display an invalid-choice message.
- The program terminates when the user selects Exit.

PROGRAM 6B: Write a python program to implement stack operations such as push, pop, peek, display using a linked list.

CODE:

6b.py - C:/Users/amuda/Desktop/DSA25124/6b.py (3.14.0)

File Edit Format Run Options Window Help

```
print("Name: A.Likitha")
print("Roll number:25124")
print("--------------------")
# Stack implementation using Linked List

class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class Stack:
    def __init__(self):
        self.top = None

    def push(self):
        value = int(input("Enter value: "))

        new_node = Node(value)
        new_node.next = self.top
        self.top = new_node

        print(value, "pushed into stack")

    def pop(self):
        if self.top is None:
            print("Stack Underflow")
        else:
            value = self.top.data
            self.top = self.top.next
            print(value, "popped from stack")

    def peek(self):
        if self.top is None:
            print("Stack is empty")
        else:
            print("Top element:", self.top.data)

    def display(self):
        if self.top is None:
            print("Stack is empty")
        else:
            temp = self.top
            print("Stack elements:")
```

```
            while temp is not None:
                print(temp.data)
                temp = temp.next


stack = Stack()

while True:
    print("\n--- STACK USING LINKED LIST ---")
    print("1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        stack.push()
    elif choice == 2:
        stack.pop()
    elif choice == 3:
        stack.peek()
    elif choice == 4:
        stack.display()
    elif choice == 5:
        break
    else:
        print("Invalid choice")
```

OUTPUT:

```
================ RESTART: C:/Users/amuda/Desktop/DSA25124/
Name: A.Likitha
Roll number:25124
--------------------

--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter value: 2
2 pushed into stack

--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter value: 15
15 pushed into stack

--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter value: 24
24 pushed into stack

--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter value: 20
20 pushed into stack
```

```
--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 3
Top element: 24

--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
Stack elements:
24
15
2

--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 2
24 popped from stack

--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 2
15 popped from stack

--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
```

```
5. Exit
Enter your choice: 2
2 popped from stack

--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 2
Stack Underflow

--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 5
```

EXPLANATION:

- The program implements a Stack using a Linked List following the LIFO (Last In, First Out) principle.
- The Node class stores data and a pointer to the next node.
- The Stack class manages the stack using the top pointer.
- The push() function inserts a new node at the top of the stack.
- The pop() function removes the top node. If the stack is empty, it displays Stack Underflow.
- The peek() function displays the top element without removing it.
- The display() function traverses the linked list and prints all stack elements from top to bottom.
- The new_node.next = self.top statement links the new node to the current top node.
- The self.top = new_node statement updates the top pointer to the newly inserted node.
- The while temp is not None loop displays each node until the end of the linked list.
- The menu-driven program allows the user to select push, pop, peek, and display operations.
- The while True loop repeats the menu until the user selects Exit.
- The program handles empty-stack conditions and invalid menu choices.

ERROR TABLE:

S.NO	ERROR	RECTIFICATION
1.	AttributeError occurred when next was accessed incorrectly.	Correctly used self.top and temp.next.
2.	IndentationError occurred because the statements inside the functions were not properly aligned.	Corrected the indentation of all code blocks.
3.	ValueError occurred when a non-integer value was entered.	Entered only valid integer values.
4.	NameError occurred because top, temp, or new_node was misspelled.	Corrected the variable names and their spelling.

WEEK-7

PROGRAM-7(A):

AIM: Write a python program to implement Queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach. Implement using array, and linked list data structures.

PROGRAM-7(A1):

Write a python program to implement Queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach. Implement using array data structures.

CODE:

7a1.py - C:/Users/amuda/Desktop/DSA25124/7a1.py (3.14.0)

File Edit Format Run Options Window Help

```
print("Name: A.Likitha")
print("Roll number:25124")
print("-------------------------")
# Queue implementation using Array (List)

queue = []
MAX_SIZE = 5


# Function to insert an element into the queue
def enqueue():
    if len(queue) == MAX_SIZE:
        print("Queue Overflow! Queue is full.")
    else:
        value = int(input("Enter the element to enqueue: "))
        queue.append(value)
        print(value, "inserted into the queue.")


# Function to delete an element from the queue
def dequeue():
    if len(queue) == 0:
        print("Queue Underflow! Queue is empty.")
    else:
        value = queue.pop(0)
        print(value, "deleted from the queue.")


# Function to display the front element
def peek():
    if len(queue) == 0:
        print("Queue is empty.")
    else:
        print("Front element is:", queue[0])

# Function to display all elements
def display():
    if len(queue) == 0:
        print("Queue is empty.")
    else:
        print("Queue elements are:")
        for element in queue:
            print(element, end=" ")
        print()
```

```python
# Menu-driven program
while True:
    print("\n----- QUEUE MENU -----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        enqueue()

    elif choice == 2:
        dequeue()

    elif choice == 3:
        peek()

    elif choice == 4:
        display()

    elif choice == 5:
        print("Program terminated.")
        break

    else:
        print("Invalid choice! Please try again.")
```

OUTPUT:

```
================ RESTART: C:/Users/amuda/D
Name: A.Likitha
Roll number:25124
-------------------------

----- QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 15
15 inserted into the queue.

----- QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 23
23 inserted into the queue.

----- QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 33
33 inserted into the queue.

----- QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 26
26 inserted into the queue.
```

```
----- QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 2
15 deleted from the queue.

----- QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 3
Front element is: 23

----- QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
Queue elements are:
23 33 26

----- QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 5
Program terminated.
```

EXPLANATION:

- The program implements a Queue using an Array (Python list) following the FIFO (First In, First Out) principle.
- queue = [] creates an empty queue, and MAX_SIZE = 5 sets its maximum capacity to five elements.
- The enqueue() function inserts an element at the rear of the queue using queue.append(). If the queue is full, it displays Queue Overflow.
- The dequeue() function removes the front element using queue.pop(0). If the queue is empty, it displays Queue Underflow.
- The peek() function displays the front element without removing it.
- The display() function prints all elements from front to rear.

- len(queue) checks the number of elements to identify whether the queue is full or empty.
- The menu-driven program allows the user to select enqueue, dequeue, peek, and display operations.
- The while True loop repeatedly displays the menu until the user selects Exit.

ERROR TABLE:

S.NO	Error	Error Type	Rectification
1.	ValueError: invalid literal for int()	Runtime Error	Enter a valid integer as input.
2.	Wrong index used for peek	Logical Error	Use queue[0] to access the front element.
3.	IndexError: pop from empty list	Runtime Error	Check whether the queue is empty before performing dequeue.

PROGRAM-7(A2):

Write a python program to implement Queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach. Implement using linked list data structures.

CODE:

7a2.py - C:/Users/amuda/Desktop/DSA25124/7a2.py (3.14.0)

File Edit Format Run Options Window Help

```
print("Name: A.Likitha")
print("Roll number:25124")
print("------------------")
# Queue implementation using Linked List


# Node class
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


# Queue class
class Queue:
    def __init__(self):
        self.front = None
        self.rear = None


    # Function to insert an element
    def enqueue(self):
        value = int(input("Enter the element to enqueue: "))

        new_node = Node(value)

        # If queue is empty
        if self.rear is None:
            self.front = self.rear = new_node

        else:
            # Add new node at the rear
            self.rear.next = new_node
            self.rear = new_node

        print(value, "inserted into the queue.")


    # Function to delete an element
    def dequeue(self):
        # If queue is empty
        if self.front is None:
            print("Queue Underflow! Queue is empty.")
            return

        value = self.front.data
```

7a2.py - C:/Users/amuda/Desktop/DSA25124/7a2.py (3.14.0)

File Edit Format Run Options Window Help

```
        value = self.front.data

        # Move front to the next node
        self.front = self.front.next

        # If queue becomes empty
        if self.front is None:
            self.rear = None

        print(value, "deleted from the queue.")


    # Function to display the front element
    def peek(self):
        if self.front is None:
            print("Queue is empty.")
        else:
            print("Front element is:", self.front.data)


    # Function to display all elements
    def display(self):
        if self.front is None:
            print("Queue is empty.")
            return

        current = self.front

        print("Queue elements are:")

        while current is not None:
            print(current.data, end=" ")
            current = current.next

        print()


# Create queue object
q = Queue()
```

```
# Menu-driven program
while True:
    print("\n----- QUEUE MENU -----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        q.enqueue()

    elif choice == 2:
        q.dequeue()

    elif choice == 3:
        q.peek()

    elif choice == 4:
        q.display()

    elif choice == 5:
        print("Program terminated.")
        break

    else:
        print("Invalid choice! Please try again.")
```

OUTPUT:

```
================ RESTART: C:/Users/amuda/Desktop/D
Name: A.Likitha
Roll number:25124
------------------

----- QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 21
21 inserted into the queue.

----- QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 26
Invalid choice! Please try again.


----- QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 27
Invalid choice! Please try again.

----- QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 31
Invalid choice! Please try again.
```

```
----- QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 2
21 deleted from the queue.

----- QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 3
Queue is empty.

----- QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
Queue is empty.

----- QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 5
Program terminated.
```

EXPLANATION:

- The program implements a Queue using a Linked List following the FIFO (First In, First Out) principle.
- The Node class stores data and a pointer to the next node.
- The Queue class manages the queue using two pointers: front and rear.
- The enqueue() function inserts a new node at the rear of the queue.
- If the queue is empty, both front and rear point to the new node.
- The dequeue() function removes the front node and moves the front pointer to the next node.
- If the queue becomes empty after deletion, rear is also set to None.
- The peek() function displays the front element without removing it.

- The display() function traverses the linked list and prints all elements from front to rear.
- The current variable is used to visit each node during traversal.
- The menu-driven program allows the user to select enqueue, dequeue, peek, and display operations.
- The while True loop repeatedly displays the menu until the user selects Exit.
- The program handles empty-queue conditions and invalid menu choices.

PROGRAM-7(B):

AIM: Write a python program to implement Circular Queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach. Implement using array, and linked list data structures.

PROGRAM-7(B1):

Write a python program to implement Circular Queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach. Implement using array data structures.

CODE:

7b1.py - C:/Users/amuda/Desktop/DSA25124/7b1.py (3.14.0)

File Edit Format Run Options Window Help

```
print("Name: A.Likitha")
print("Roll number:25124")
print("--------------------")
# Circular Queue implementation using Array

MAX_SIZE = 5

queue = [None] * MAX_SIZE
front = -1
rear = -1


# Function to insert an element
def enqueue():
    global front, rear

    # Check if queue is full
    if (rear + 1) % MAX_SIZE == front:
        print("Circular Queue is Full!")
        return

    value = int(input("Enter the element to enqueue: "))

    # If queue is empty
    if front == -1:
        front = 0
        rear = 0
    else:
        # Move rear circularly
        rear = (rear + 1) % MAX_SIZE

    queue[rear] = value

    print(value, "inserted into the queue.")


# Function to delete an element
def dequeue():
    global front, rear

    # Check if queue is empty
    if front == -1:
        print("Circular Queue is Empty!")
        return

    value = queue[front]
```

7b1.py - C:/Users/amuda/Desktop/DSA25124/7b1.py (3.14.0)

File Edit Format Run Options Window Help

```
    value = queue[front]

    # If only one element is present
    if front == rear:
        front = -1
        rear = -1
    else:
        # Move front circularly
        front = (front + 1) % MAX_SIZE

    print(value, "deleted from the queue.")


# Function to display front element
def peek():
    if front == -1:
        print("Circular Queue is Empty!")
    else:
        print("Front element is:", queue[front])


# Function to display all elements
def display():
    if front == -1:
        print("Circular Queue is Empty!")
        return

    print("Circular Queue elements are:")

    i = front

    while True:
        print(queue[i], end=" ")

        if i == rear:
            break

        i = (i + 1) % MAX_SIZE

    print()
```

```
# Menu-driven program
while True:

    print("\n----- CIRCULAR QUEUE MENU -----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        enqueue()

    elif choice == 2:
        dequeue()

    elif choice == 3:
        peek()

    elif choice == 4:
        display()

    elif choice == 5:
        print("Program terminated.")
        break

    else:
        print("Invalid choice! Please try again.")
```

OUTPUT:

```
================ RESTART: C:/Users/amuda
Name: A.Likitha
Roll number:25124
--------------------

----- CIRCULAR QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 12
12 inserted into the queue.

----- CIRCULAR QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 34
34 inserted into the queue.

----- CIRCULAR QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 54
54 inserted into the queue.

----- CIRCULAR QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 26
26 inserted into the queue.
```

```
----- CIRCULAR QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 2
12 deleted from the queue.

----- CIRCULAR QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 3
Front element is: 34

----- CIRCULAR QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
Circular Queue elements are:
34 54 26

----- CIRCULAR QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 5
Program terminated.
```

EXPLANTION:

- The program implements a Circular Queue using an Array and follows the FIFO (First In, First Out) principle.
- MAX_SIZE = 5 sets the maximum capacity of the queue.
- queue = [None] * MAX_SIZE creates an array with five empty positions.
- front points to the first element, while rear points to the last element. Both are initialized to -1, indicating an empty queue.
- The enqueue() function inserts an element at the rear and checks whether the queue is full.

- (rear + 1) % MAX_SIZE == front checks the circular queue overflow condition.
- The dequeue() function removes the front element and updates the front pointer.
- If the last element is removed, both front and rear are reset to -1.
- The peek() function displays the front element without removing it.
- The display() function prints all elements from front to rear, including when the queue wraps around.
- The modulo operator % moves the front and rear pointers circularly to reuse empty array positions.
- The global keyword allows the enqueue() and dequeue() functions to modify front and rear.
- The menu-driven program allows the user to select enqueue, dequeue, peek, and display operations until Exit is selected.

PROGRAM-7(B2):

Write a python program to implement Circular Queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach. Implement using array, and linked list data structures.

CODE:

7b2.py - C:/Users/amuda/Desktop/DSA25124/7b2.py (3.14.0)

File Edit Format Run Options Window Help

```
print("Name: A.Likitha")
print("Roll number:25124")
print("----------------------")
# Circular Queue implementation using Linked List


# Node class
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


# Circular Queue class
class CircularQueue:

    def __init__(self):
        self.front = None
        self.rear = None


    # Function to insert an element
    def enqueue(self):
        value = int(input("Enter the element to enqueue: "))

        new_node = Node(value)

        # If queue is empty
        if self.front is None:

            self.front = new_node
            self.rear = new_node

            # Make the queue circular
            self.rear.next = self.front

        else:
            # Add new node after rear
            self.rear.next = new_node

            # Move rear to new node
            self.rear = new_node

            # Maintain circular connection
            self.rear.next = self.front
```

7b2.py - C:/Users/amuda/Desktop/DSA25124/7b2.py (3.14.0)

File Edit Format Run Options Window Help

```
        print(value, "inserted into the queue.")


    # Function to delete an element
    def dequeue(self):

        # Check if queue is empty
        if self.front is None:
            print("Circular Queue is Empty!")
            return

        value = self.front.data

        # If only one node exists
        if self.front == self.rear:
            self.front = None
            self.rear = None

        else:
            # Move front to next node
            self.front = self.front.next

            # Maintain circular connection
            self.rear.next = self.front

        print(value, "deleted from the queue.")


    # Function to display front element
    def peek(self):

        if self.front is None:
            print("Circular Queue is Empty!")
        else:
            print("Front element is:", self.front.data)


    # Function to display all elements
    def display(self):

        if self.front is None:
            print("Circular Queue is Empty!")
            return

        print("Circular Queue elements are:")
```

7b2.py - C:/Users/amuda/Desktop/DSA25124/7b2.py (3.14.0)

File Edit Format Run Options Window Help

```
        print("Circular Queue elements are:")

        current = self.front

        while True:
            print(current.data, end=" ")

            current = current.next

            if current == self.front:
                break

        print()


# Create Circular Queue object
q = CircularQueue()


# Menu-driven program
while True:

    print("\n----- CIRCULAR QUEUE MENU -----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        q.enqueue()

    elif choice == 2:
        q.dequeue()

    elif choice == 3:
        q.peek()

    elif choice == 4:
        q.display()

    elif choice == 5:
        print("Program terminated.")
        break
```

OUTPUT:

```
================ RESTART: C:/Users/amuda/Des
Name: A.Likitha
Roll number:25124
---------------------

----- CIRCULAR QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 15
15 inserted into the queue.

----- CIRCULAR QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 24
24 inserted into the queue.

----- CIRCULAR QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 35
35 inserted into the queue.

----- CIRCULAR QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 25
25 inserted into the queue.
```

```
5. Exit
Enter your choice: 1
Enter the element to enqueue: 25
25 inserted into the queue.

----- CIRCULAR QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 2
15 deleted from the queue.

----- CIRCULAR QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 3
Front element is: 24

----- CIRCULAR QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
Circular Queue elements are:
24 35 25

----- CIRCULAR QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 5
Program terminated.
```

ERROR TABLE:

S.NO	Error	Error Type	Rectification
1.	Indentation Error in line 2	Syntax Error	Use proper indentation inside the class and its methods.
2.	Display loop becomes infinite	Logical Error	Stop the traversal when current == front.
3.	Last node does not point to front	Logical Error	Set rear.next = front after insertion.

EXPLANATION:

- The program implements a Circular Queue using a Linked List and follows the FIFO (First In, First Out) principle.
- The Node class stores data and a pointer to the next node.
- The CircularQueue class manages the queue using two pointers: front and rear.
- The enqueue() function inserts a new node at the rear of the queue.
- If the queue is empty, both front and rear point to the new node.
- The rear.next = front statement maintains the circular connection between the last node and the first node.
- The dequeue() function removes the front element and updates the front pointer.
- If only one node exists, both front and rear are set to None.
- The peek() function displays the front element without removing it.
- The display() function traverses the queue and prints all elements until it reaches the front again.
- The while True loop in the display() function ensures that every node is displayed exactly once.
- The menu-driven program allows the user to select enqueue, dequeue, peek, and display operations.
- The program handles empty-queue conditions and invalid menu choices.

WEEK-8

PROGRAM-8(A):

AIM: Write a python program to implement a binary tree with its operations using an array.

CODE:

*8a.py - C:/Users/amuda/AppData/Local/Programs/Python/Python314/8a.py

File Edit Format Run Options Window Help

```
class BinaryTree:
    def __init__(self):
        self.tree = []

    def insert(self, data):
        self.tree.append(data)
        print("Node inserted successfully")

    def preorder(self, i=0):
        if i < len(self.tree):
            print(self.tree[i], end=" ")
            self.preorder(2 * i + 1)
            self.preorder(2 * i + 2)

    def inorder(self, i=0):
        if i < len(self.tree):
            self.inorder(2 * i + 1)
            print(self.tree[i], end=" ")
            self.inorder(2 * i + 2)

    def postorder(self, i=0):
        if i < len(self.tree):
            self.postorder(2 * i + 1)
            self.postorder(2 * i + 2)
            print(self.tree[i], end=" ")

    def search(self, key):
        if key in self.tree:
            print("Element found")
        else:
            print("Element not found")

    def delete(self, key):
        if key in self.tree:
            i = self.tree.index(key)
            last = self.tree.pop()
            if i < len(self.tree):
                self.tree[i] = last
            print("Node deleted successfully")
        else:
            print("Element not found")

    def display(self):
        print("Array:", self.tree)
```

8a.py - C:/Users/amuda/AppData/Local/Programs/Python/Python314/8a.py (3.14.0)

File Edit Format Run Options Window Help

```
bt = BinaryTree()

while True:
    print("\n1. Insert")
    print("2. Preorder Traversal")
    print("3. Inorder Traversal")
    print("4. Postorder Traversal")
    print("5. Search")
    print("6. Delete")
    print("7. Display")
    print("8. Exit")

    try:
        choice = int(input("Enter your choice: "))

        if choice == 1:
            data = int(input("Enter data: "))
            bt.insert(data)
        elif choice == 2:
            print("Preorder:", end=" ")
            bt.preorder()
            print()
        elif choice == 3:
            print("Inorder:", end=" ")
            bt.inorder()
            print()
        elif choice == 4:
            print("Postorder:", end=" ")
            bt.postorder()
            print()
        elif choice == 5:
            key = int(input("Enter element to search: "))
            bt.search(key)
        elif choice == 6:
            key = int(input("Enter element to delete: "))
            bt.delete(key)
        elif choice == 7:
            bt.display()
        elif choice == 8:
            print("Exiting program")
            break
        else:
            print("Invalid choice")

    except ValueError:
        print("Please enter a valid integer")
```

OUTPUT:

```
================ RESTART: C:/Users/am
Name: A.Likitha
Roll number:25124
-------------------------

1. Insert
2. Preorder Traversal
3. Inorder Traversal
4. Postorder Traversal
5. Search
6. Delete
7. Display
8. Exit
Enter your choice: 1
Enter data: 12
Node inserted successfully

1. Insert
2. Preorder Traversal
3. Inorder Traversal
4. Postorder Traversal
5. Search
6. Delete
7. Display
8. Exit
Enter your choice: 1
Enter data: 13
Node inserted successfully

1. Insert
2. Preorder Traversal
3. Inorder Traversal
4. Postorder Traversal
5. Search
6. Delete
7. Display
8. Exit
Enter your choice: 1
Enter data: 15
Node inserted successfully

1. Insert
2. Preorder Traversal
3. Inorder Traversal
```

```
2. Preorder Traversal
3. Inorder Traversal
4. Postorder Traversal
5. Search
6. Delete
7. Display
8. Exit
Enter your choice: 1
Enter data: 26
Node inserted successfully

1. Insert
2. Preorder Traversal
3. Inorder Traversal
4. Postorder Traversal
5. Search
6. Delete
7. Display
8. Exit
Enter your choice: 1
Enter data: 35
Node inserted successfully

1. Insert
2. Preorder Traversal
3. Inorder Traversal
4. Postorder Traversal
5. Search
6. Delete
7. Display
8. Exit
Enter your choice: 2
Preorder: 12 13 26 35 15

1. Insert
2. Preorder Traversal
3. Inorder Traversal
4. Postorder Traversal
5. Search
6. Delete
7. Display
8. Exit
Enter your choice: 3
Inorder: 26 13 35 12 15
```

```
1. Insert
2. Preorder Traversal
3. Inorder Traversal
4. Postorder Traversal
5. Search
6. Delete
7. Display
8. Exit
Enter your choice: 4
Postorder: 26 35 13 15 12

1. Insert
2. Preorder Traversal
3. Inorder Traversal
4. Postorder Traversal
5. Search
6. Delete
7. Display
8. Exit
Enter your choice: 5
Enter element to search: 35
Element found

1. Insert
2. Preorder Traversal
3. Inorder Traversal
4. Postorder Traversal
5. Search
6. Delete
7. Display
8. Exit
Enter your choice: 6
Enter element to delete: 12
Node deleted successfully

1. Insert
2. Preorder Traversal
3. Inorder Traversal
4. Postorder Traversal
5. Search
6. Delete
7. Display
8. Exit
Enter your choice: 7
Array: [35, 13, 15, 26]


1. Insert
2. Preorder Traversal
3. Inorder Traversal
4. Postorder Traversal
5. Search
6. Delete
7. Display
8. Exit
Enter your choice: 8
Exiting program
```

ERROR TABLE:

S.NO	ERROR	RECTIFICATION
1.	Incorrect child index occurred when the left or right child position was calculated incorrectly..	Used 2 * parent_index + 1 for the left child and 2 * parent_index + 2 for the right child..
2.	Parent node error occurred when trying to insert a child for a parent that does not exist.	Checked whether self.tree[parent_index] is None before insertion.
3.	Incorrect traversal output occurred when the preorder, inorder, or postorder sequence was not followed correctly.	Used the correct traversal orders: Root–Left–Right, Left–Root–Right, and Left–Right–Root.
4.	IndexError may occur if an invalid index is accessed in the tree array.	Checked the index range before accessing the array.

EXPLANATION:

• The BinaryTree class creates a binary tree using a Python list (array).

• The __init__() function initializes an empty list to store the tree elements.

• The insert() function adds a new element to the end of the array.

• The preorder() function traverses the tree in the order Root → Left → Right.

• The inorder() function traverses the tree in the order Left → Root → Right.

• The postorder() function traverses the tree in the order Left → Right → Root.

• The search() function checks whether the specified element exists in the array.

• The delete() function removes an element by replacing it with the last element in the array.

• The display() function displays the elements stored in the array.

• The main menu allows the user to select and perform the required binary tree operations.

• The try-except block handles invalid inputs and displays an error message if the user enters a non-integer value.

• The program continues executing until the user selects the Exit option.

PROGRAM-8(B):

AIM: Write a python program to implement a binary tree with its operations using a linked list.

CODE:

8b.py - C:/Users/amuda/Desktop/DSA25124/8b.py (3.14.0)

File Edit Format Run Options Window Help

```
print("Name: A.Likitha")
print("Roll number: 25124")
print("------------------------")
from collections import deque

class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None


class BinaryTree:
    def __init__(self):
        self.root = None

    def insert(self, data):
        new_node = Node(data)

        if self.root is None:
            self.root = new_node
            print("Node inserted successfully")
            return

        queue = deque([self.root])

        while queue:
            current = queue.popleft()

            if current.left is None:
                current.left = new_node
                print("Node inserted successfully")
                return
            else:
                queue.append(current.left)

            if current.right is None:
                current.right = new_node
                print("Node inserted successfully")
                return
            else:
                queue.append(current.right)
```

8b.py - C:/Users/amuda/Desktop/DSA25124/8b.py (3.14.0)

File Edit Format Run Options Window Help

```
            else:
                queue.append(current.right)

    def preorder(self, node):
        if node:
            print(node.data, end=" ")
            self.preorder(node.left)
            self.preorder(node.right)

    def inorder(self, node):
        if node:
            self.inorder(node.left)
            print(node.data, end=" ")
            self.inorder(node.right)

    def postorder(self, node):
        if node:
            self.postorder(node.left)
            self.postorder(node.right)
            print(node.data, end=" ")

    def search(self, key):
        if self.root is None:
            return False

        queue = deque([self.root])

        while queue:
            current = queue.popleft()

            if current.data == key:
                return True

            if current.left:
                queue.append(current.left)

            if current.right:
                queue.append(current.right)

        return False

    def delete(self, key):
        if self.root is None:
            print("Tree is empty")
            return
```

8b.py - C:/Users/amuda/Desktop/DSA25124/8b.py (3.14.0)

File Edit Format Run Options Window Help

```
        queue = deque([(self.root, None)])
        target = None
        last = None
        last_parent = None

        while queue:
            current, parent = queue.popleft()

            if current.data == key and target is None:
                target = current

            last = current
            last_parent = parent

            if current.left:
                queue.append((current.left, current))

            if current.right:
                queue.append((current.right, current))

        if target is None:
            print("Element not found")
            return

        target.data = last.data

        if last_parent is None:
            self.root = None
        elif last_parent.right == last:
            last_parent.right = None
        else:
            last_parent.left = None

        print("Node deleted successfully")

    def level_order(self):
        if self.root is None:
            print("Tree is empty")
            return

        queue = deque([self.root])

        while queue:
            current = queue.popleft()
            print(current.data, end=" ")
```

8b.py - C:/Users/amuda/Desktop/DSA25124/8b.py (3.14.0)

File Edit Format Run Options Window Help

```
        while queue:
            current = queue.popleft()
            print(current.data, end=" ")

            if current.left:
                queue.append(current.left)

            if current.right:
                queue.append(current.right)


# Main program
bt = BinaryTree()

while True:
    print("\n\n1. Insert")
    print("2. Preorder Traversal")
    print("3. Inorder Traversal")
    print("4. Postorder Traversal")
    print("5. Search")
    print("6. Delete")
    print("7. Level Order Traversal")
    print("8. Exit")

    try:
        choice = int(input("Enter your choice: "))

        if choice == 1:
            data = int(input("Enter data: "))
            bt.insert(data)

        elif choice == 2:
            print("Preorder:", end=" ")
            bt.preorder(bt.root)
            print()

        elif choice == 3:
            print("Inorder:", end=" ")
            bt.inorder(bt.root)
            print()

        elif choice == 4:
            print("Postorder:", end=" ")
            bt.postorder(bt.root)
            print()

        elif choice == 5:
```

```
        elif choice == 5:
            key = int(input("Enter element to search: "))
            if bt.search(key):
                print("Element found")
            else:
                print("Element not found")

        elif choice == 6:
            key = int(input("Enter element to delete: "))
            bt.delete(key)

        elif choice == 7:
            print("Level order:", end=" ")
            bt.level_order()
            print()

        elif choice == 8:
            print("Exiting program")
            break

        else:
            print("Invalid choice")

    except ValueError:
        print("Please enter a valid integer")
```

OUTPUT:

```
================ RESTART: C:/Users/an
Name: A.Likitha
Roll number: 25124
-----------------------


1. Insert
2. Preorder Traversal
3. Inorder Traversal
4. Postorder Traversal
5. Search
6. Delete
7. Level Order Traversal
8. Exit
Enter your choice: 1
Enter data: 12
Node inserted successfully


1. Insert
2. Preorder Traversal
3. Inorder Traversal
4. Postorder Traversal
5. Search
6. Delete
7. Level Order Traversal
8. Exit
Enter your choice: 1
Enter data: 15
Node inserted successfully


1. Insert
2. Preorder Traversal
3. Inorder Traversal
4. Postorder Traversal
5. Search
6. Delete
7. Level Order Traversal
8. Exit
Enter your choice: 1
Enter data: 34
Node inserted successfully
```

```
3. Inorder Traversal
4. Postorder Traversal
5. Search
6. Delete
7. Level Order Traversal
8. Exit
Enter your choice: 1
Enter data: 556
Node inserted successfully


1. Insert
2. Preorder Traversal
3. Inorder Traversal
4. Postorder Traversal
5. Search
6. Delete
7. Level Order Traversal
8. Exit
Enter your choice: 1
Enter data: 45
Node inserted successfully


1. Insert
2. Preorder Traversal
3. Inorder Traversal
4. Postorder Traversal
5. Search
6. Delete
7. Level Order Traversal
8. Exit
Enter your choice: 2
Preorder: 12 15 556 45 34


1. Insert
2. Preorder Traversal
3. Inorder Traversal
4. Postorder Traversal
5. Search
6. Delete
7. Level Order Traversal
8. Exit
Enter your choice: 3
Inorder: 556 15 45 12 34
```

```
3. Inorder Traversal
4. Postorder Traversal
5. Search
6. Delete
7. Level Order Traversal
8. Exit
Enter your choice: 4
Postorder: 556 45 15 34 12


1. Insert
2. Preorder Traversal
3. Inorder Traversal
4. Postorder Traversal
5. Search
6. Delete
7. Level Order Traversal
8. Exit
Enter your choice: 5
Enter element to search: 12
Element found


1. Insert
2. Preorder Traversal
3. Inorder Traversal
4. Postorder Traversal
5. Search
6. Delete
7. Level Order Traversal
8. Exit
Enter your choice: 6
Enter element to delete: 556
Node deleted successfully


1. Insert
2. Preorder Traversal
3. Inorder Traversal
4. Postorder Traversal
5. Search
6. Delete
7. Level Order Traversal
8. Exit
Enter your choice: 7
Level order: 12 15 34 45
```

```
1. Insert
2. Preorder Traversal
3. Inorder Traversal
4. Postorder Traversal
5. Search
6. Delete
7. Level Order Traversal
8. Exit
Enter your choice: 8
Exiting program
```

ERROR TABLE:

S.NO	ERROR	RECTIFICATION
1.	Incorrect traversal output occurred when traversal order was incorrect.	Followed the correct preorder, inorder, postorder, and level-order sequences.
2.	Root already exists error occurred when trying to insert another root.	Checked whether self.root is None before insertion.
3.	AttributeError occurred when accessing node links incorrectly.	Used node.left and node.right correctly.
4.	Child already exists error occurred when inserting a child into an occupied position.	Checked whether node.left or node.right is None before insertion.

EXPLANATION:

• The Node class creates a node containing data, a left link, and a right link.

• The BinaryTree class manages the binary tree using linked nodes.

• The __init__() function initializes the root of the binary tree as None.

• The insert() function inserts a new node at the first available position using level-order traversal.

• The preorder() function traverses the tree in the order Root → Left → Right.

• The inorder() function traverses the tree in the order Left → Root → Right.

• The postorder() function traverses the tree in the order Left → Right → Root.

• The search() function searches for a specified element by visiting the nodes level by level.

• The delete() function deletes a node by replacing its value with the deepest, rightmost node's value and removing that last node.

• The level_order() function visits the nodes level by level using a queue.

• The main menu allows the user to select and perform the required binary tree operations.

• The try-except block handles invalid inputs and displays an error message when a non-integer value is entered.

• The program continues executing until the user selects the Exit option.

WEEK-9

PROGRAM-9:

AIM: Write a python to implement a Binary Search Tree with its operations traversals.

CODE:

BST.py - C:/Users/amuda/Desktop/DSA/BST.py (3.14.0)

File Edit Format Run Options Window Help

```
    def find_max(self, root):
        if root is None:
            return None
        while root.right:
            root = root.right
        return root.data

    def search(self, root, key):
        if root is None or root.data == key:
            return root
        if key < root.data:
            return self.search(root.left, key)
        return self.search(root.right, key)

    def delete(self, root, key):
        if root is None:
            return None

        if key < root.data:
            root.left = self.delete(root.left, key)
        elif key > root.data:
            root.right = self.delete(root.right, key)
        else:
            if root.left is None:
                return root.right
            if root.right is None:
                return root.left

            successor = root.right
            while successor.left:
                successor = successor.left

            root.data = successor.data
            root.right = self.delete(root.right, successor.data)

        return root

    def count_nodes(self, root):
        if root is None:
            return 0
        return 1 + self.count_nodes(root.left) + self.count_nodes(root.right)
```

BST.py - C:/Users/amuda/Desktop/DSA/BST.py (3.14.0)

File Edit Format Run Options Window Help

```
        return 1 + self.count_nodes(root.left) + self.count_nodes(root.right)

    def count_leaf_nodes(self, root):
        if root is None:
            return 0
        if root.left is None and root.right is None:
            return 1
        return (self.count_leaf_nodes(root.left) +
                self.count_leaf_nodes(root.right))

    def count_internal_nodes(self, root):
        if root is None or (root.left is None and root.right is None):
            return 0
        return (1 + self.count_internal_nodes(root.left) +
                self.count_internal_nodes(root.right))

    def height(self, root):
        if root is None:
            return 0
        return 1 + max(self.height(root.left), self.height(root.right))

    def level_order(self, root):
        if root is None:
            return
        queue = deque([root])
        while queue:
            node = queue.popleft()
            print(node.data, end=" ")
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
```

BST.py - C:/Users/amuda/Desktop/DSA/BST.py (3.14.0)

File Edit Format Run Options Window Help

```
            if node.right:
                queue.append(node.right)


# Main program
tree = BST()
root = None

while True:
    print("\n\n1. Insert")
    print("2. Preorder Traversal")
    print("3. Inorder Traversal")
    print("4. Postorder Traversal")
    print("5. Find Minimum")
    print("6. Find Maximum")
    print("7. Search")
    print("8. Delete")
    print("9. Count Total Nodes")
    print("10. Count Leaf Nodes")
    print("11. Count Internal Nodes")
    print("12. Find Height")
    print("13. Level Order Traversal")
    print("14. Exit")

    try:
        choice = int(input("Enter your choice: "))

        if choice == 1:
            data = int(input("Enter data: "))
            root = tree.insert(root, data)
            print("Node inserted successfully")

        elif choice == 2:
            print("Preorder:", end=" ")
            tree.preorder(root)
            print()

        elif choice == 3:
            print("Inorder:", end=" ")
            tree.inorder(root)
            print()

        elif choice == 4:
            print("Postorder:", end=" ")
            tree.postorder(root)
            print()
```

BST.py - C:/Users/amuda/Desktop/DSA/BST.py (3.14.0)

File Edit Format Run Options Window Help

```
            print()

        elif choice == 5:
            result = tree.find_min(root)
            print("Minimum:", result if result is not None
                  else "Tree is empty")

        elif choice == 6:
            result = tree.find_max(root)
            print("Maximum:", result if result is not None
                  else "Tree is empty")

        elif choice == 7:
            key = int(input("Enter value to search: "))
            if tree.search(root, key):
                print("Element found")
            else:
                print("Element not found")

        elif choice == 8:
            key = int(input("Enter value to delete: "))
            if tree.search(root, key):
                root = tree.delete(root, key)
                print("Node deleted successfully")
            else:
                print("Element not found")

        elif choice == 9:
            print("Total nodes:", tree.count_nodes(root))

        elif choice == 10:
            print("Leaf nodes:", tree.count_leaf_nodes(root))

        elif choice == 11:
            print("Internal nodes:", tree.count_internal_nodes(root))

        elif choice == 12:
            print("Height:", tree.height(root))

        elif choice == 13:
            print("Level order:", end=" ")
            tree.level_order(root)
            print()

        elif choice == 14:
            print("Exiting program")
            break
```

BST.py - C:/Users/amuda/Desktop/DSA/BST.py (3.14.0)

File Edit Format Run Options Window Help

```
        elif choice == 6:
            result = tree.find_max(root)
            print("Maximum:", result if result is not None
                  else "Tree is empty")

        elif choice == 7:
            key = int(input("Enter value to search: "))
            if tree.search(root, key):
                print("Element found")
            else:
                print("Element not found")

        elif choice == 8:
            key = int(input("Enter value to delete: "))
            if tree.search(root, key):
                root = tree.delete(root, key)
                print("Node deleted successfully")
            else:
                print("Element not found")

        elif choice == 9:
            print("Total nodes:", tree.count_nodes(root))

        elif choice == 10:
            print("Leaf nodes:", tree.count_leaf_nodes(root))

        elif choice == 11:
            print("Internal nodes:", tree.count_internal_nodes(root))

        elif choice == 12:
            print("Height:", tree.height(root))

        elif choice == 13:
            print("Level order:", end=" ")
            tree.level_order(root)
            print()

        elif choice == 14:
            print("Exiting program")
            break

        else:
            print("Invalid choice")

    except ValueError:
        print("Please enter a valid integer")
```

OUTPUT:

```
================= RESTART: C:/Users/amuda/Desktop/DSA25124/9.
Name: A.Likitha
Roll number: 25124
----------------------


1. Insert
2. Preorder Traversal
3. Inorder Traversal
4. Postorder Traversal
5. Find Minimum
6. Find Maximum
7. Search
8. Delete
9. Count Total Nodes
10. Count Leaf Nodes
11. Count Internal Nodes
12. Find Height
13. Level Order Traversal
14. Exit
Enter your choice: 1
Enter data: 10
Node inserted successfully


1. Insert
2. Preorder Traversal
3. Inorder Traversal
4. Postorder Traversal
5. Find Minimum
6. Find Maximum
7. Search
8. Delete
9. Count Total Nodes
10. Count Leaf Nodes
11. Count Internal Nodes
12. Find Height
13. Level Order Traversal
14. Exit
Enter your choice: 1
Enter data: 20
Node inserted successfully
```

```
4. Postorder Traversal
5. Find Minimum
6. Find Maximum
7. Search
8. Delete
9. Count Total Nodes
10. Count Leaf Nodes
11. Count Internal Nodes
12. Find Height
13. Level Order Traversal
14. Exit
Enter your choice: 1
Enter data: 30
Node inserted successfully


1. Insert
2. Preorder Traversal
3. Inorder Traversal
4. Postorder Traversal
5. Find Minimum
6. Find Maximum
7. Search
8. Delete
9. Count Total Nodes
10. Count Leaf Nodes
11. Count Internal Nodes
12. Find Height
13. Level Order Traversal
14. Exit
Enter your choice: 1
Enter data: 40
Node inserted successfully


1. Insert
2. Preorder Traversal
3. Inorder Traversal
4. Postorder Traversal
5. Find Minimum
6. Find Maximum
7. Search
8. Delete
9. Count Total Nodes
10. Count Leaf Nodes
11. Count Internal Nodes
12. Find Height
```

```
1. Insert
2. Preorder Traversal
3. Inorder Traversal
4. Postorder Traversal
5. Find Minimum
6. Find Maximum
7. Search
8. Delete
9. Count Total Nodes
10. Count Leaf Nodes
11. Count Internal Nodes
12. Find Height
13. Level Order Traversal
14. Exit
Enter your choice: 1
Enter data: 50
Node inserted successfully


1. Insert
2. Preorder Traversal
3. Inorder Traversal
4. Postorder Traversal
5. Find Minimum
6. Find Maximum
7. Search
8. Delete
9. Count Total Nodes
10. Count Leaf Nodes
11. Count Internal Nodes
12. Find Height
13. Level Order Traversal
14. Exit
Enter your choice: 2
Preorder: 10 20 30 40 50


1. Insert
2. Preorder Traversal
3. Inorder Traversal
4. Postorder Traversal
5. Find Minimum
6. Find Maximum
7. Search
8. Delete
9. Count Total Nodes
10. Count Leaf Nodes
```

```
1. Insert
2. Preorder Traversal
3. Inorder Traversal
4. Postorder Traversal
5. Find Minimum
6. Find Maximum
7. Search
8. Delete
9. Count Total Nodes
10. Count Leaf Nodes
11. Count Internal Nodes
12. Find Height
13. Level Order Traversal
14. Exit
Enter your choice: 3
Inorder: 10 20 30 40 50


1. Insert
2. Preorder Traversal
3. Inorder Traversal
4. Postorder Traversal
5. Find Minimum
6. Find Maximum
7. Search
8. Delete
9. Count Total Nodes
10. Count Leaf Nodes
11. Count Internal Nodes
12. Find Height
13. Level Order Traversal
14. Exit
Enter your choice: 4
Postorder: 50 40 30 20 10


1. Insert
2. Preorder Traversal
3. Inorder Traversal
4. Postorder Traversal
5. Find Minimum
6. Find Maximum
7. Search
8. Delete
9. Count Total Nodes
```

```
8. Delete
9. Count Total Nodes
10. Count Leaf Nodes
11. Count Internal Nodes
12. Find Height
13. Level Order Traversal
14. Exit
Enter your choice: 5
Minimum: 10


1. Insert
2. Preorder Traversal
3. Inorder Traversal
4. Postorder Traversal
5. Find Minimum
6. Find Maximum
7. Search
8. Delete
9. Count Total Nodes
10. Count Leaf Nodes
11. Count Internal Nodes
12. Find Height
13. Level Order Traversal
14. Exit
Enter your choice: 6
Maximum: 50


1. Insert
2. Preorder Traversal
3. Inorder Traversal
4. Postorder Traversal
5. Find Minimum
6. Find Maximum
7. Search
8. Delete
9. Count Total Nodes
10. Count Leaf Nodes
11. Count Internal Nodes
12. Find Height
13. Level Order Traversal
14. Exit
Enter your choice: 7
Enter value to search: 8
Element not found
```

```
3. Inorder Traversal
4. Postorder Traversal
5. Find Minimum
6. Find Maximum
7. Search
8. Delete
9. Count Total Nodes
10. Count Leaf Nodes
11. Count Internal Nodes
12. Find Height
13. Level Order Traversal
14. Exit
Enter your choice: 9
Total nodes: 5


1. Insert
2. Preorder Traversal
3. Inorder Traversal
4. Postorder Traversal
5. Find Minimum
6. Find Maximum
7. Search
8. Delete
9. Count Total Nodes
10. Count Leaf Nodes
11. Count Internal Nodes
12. Find Height
13. Level Order Traversal
14. Exit
Enter your choice: 10
Leaf nodes: 1


1. Insert
2. Preorder Traversal
3. Inorder Traversal
4. Postorder Traversal
5. Find Minimum
6. Find Maximum
7. Search
8. Delete
9. Count Total Nodes
10. Count Leaf Nodes
11. Count Internal Nodes
12. Find Height
13. Level Order Traversal
```

```
5. Find Minimum
6. Find Maximum
7. Search
8. Delete
9. Count Total Nodes
10. Count Leaf Nodes
11. Count Internal Nodes
12. Find Height
13. Level Order Traversal
14. Exit
Enter your choice: 11
Internal nodes: 4


1. Insert
2. Preorder Traversal
3. Inorder Traversal
4. Postorder Traversal
5. Find Minimum
6. Find Maximum
7. Search
8. Delete
9. Count Total Nodes
10. Count Leaf Nodes
11. Count Internal Nodes
12. Find Height
13. Level Order Traversal
14. Exit
Enter your choice: 12
Height: 5


1. Insert
2. Preorder Traversal
3. Inorder Traversal
4. Postorder Traversal
5. Find Minimum
6. Find Maximum
7. Search
8. Delete
9. Count Total Nodes
10. Count Leaf Nodes
11. Count Internal Nodes
12. Find Height
13. Level Order Traversal
14. Exit
Enter your choice: 13
```

```
14. Exit
Enter your choice: 13
Level order: 10 20 30 40 50


1. Insert
2. Preorder Traversal
3. Inorder Traversal
4. Postorder Traversal
5. Find Minimum
6. Find Maximum
7. Search
8. Delete
9. Count Total Nodes
10. Count Leaf Nodes
11. Count Internal Nodes
12. Find Height
13. Level Order Traversal
14. Exit
Enter your choice: 14
Exiting program
```

ERROR TABLE:

S.NO	ERROR	RECTIFICATION
1.	TypeError/ValueError occurred when invalid input was entered.	Used try-except to validate integer inputs.
2.	Duplicate value error occurred when inserting an existing value.	Checked for duplicate values before insertion.
3.	Search error occurred when the element was not present in the BST.	Checked whether search() returns None before displaying the result.
4.	Empty tree error occurred when finding the minimum or maximum of an empty tree.	Checked whether the root is None before performing the operation.
5.	Incorrect traversal output occurred when the traversal order was incorrect.	Followed the correct preorder, inorder, postorder, and level-order sequences.

EXPLANATION:

• The Node class creates a node containing data, a left link, and a right link.

• The insert() function inserts values into the BST, placing smaller values on the left and larger values on the right.

• The preorder() function traverses the tree in the order Root → Left → Right.

• The inorder() function traverses the tree in the order Left → Root → Right.

• The postorder() function traverses the tree in the order Left → Right → Root.

• The find_min() function finds the minimum element by moving to the leftmost node.

• The find_max() function finds the maximum element by moving to the rightmost node.

• The search() function searches for an element using BST comparisons.

• The delete() function removes a node, handling cases with no children, one child, or two children.

• The count_nodes() function counts the total number of nodes in the tree.

• The count_leaf_nodes() function counts nodes that have no children.

• The count_internal_nodes() function counts nodes that have at least one child.

• The height() function calculates the height of the tree based on the longest path from the root to a leaf. In this program, height is measured in levels.

• The level_order() function visits nodes level by level using a queue.

• The main menu allows the user to select and perform each BST operation, validate inputs, and exit the program.

• The program demonstrates BST insertion, traversal, searching, deletion, minimum and maximum finding, node counting, height calculation, and level-order traversal.

WEEK-10

PROGRAM-10:

AIM: Write a Python program to implement an AVL Tree with all its operations.

CODE:

10.py - C:/Users/amuda/Desktop/DSA25124/10.py (3.14.0)

File Edit Format Run Options Window Help

```
print("Name: A.Likitha")
print("Roll number:25124")
print("--------------------")
class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None
        self.height = 1


class AVLTree:

    def height(self, root):
        if root is None:
            return 0
        return root.height

    def get_balance(self, root):
        if root is None:
            return 0
        return self.height(root.left) - self.height(root.right)

    def right_rotate(self, y):
        x = y.left
        T2 = x.right

        x.right = y
        y.left = T2

        y.height = 1 + max(self.height(y.left),
                           self.height(y.right))
        x.height = 1 + max(self.height(x.left),
                           self.height(x.right))

        return x

    def left_rotate(self, x):
        y = x.right
        T2 = y.left

        y.left = x
        x.right = T2
```

10.py - C:/Users/amuda/Desktop/DSA25124/10.py (3.14.0)

File Edit Format Run Options Window Help

```
        y.left = x
        x.right = T2

        x.height = 1 + max(self.height(x.left),
                           self.height(x.right))
        y.height = 1 + max(self.height(y.left),
                           self.height(y.right))

        return y

    def insert(self, root, key):
        if root is None:
            return Node(key)

        if key < root.key:
            root.left = self.insert(root.left, key)
        elif key > root.key:
            root.right = self.insert(root.right, key)
        else:
            return root

        root.height = 1 + max(self.height(root.left),
                              self.height(root.right))

        balance = self.get_balance(root)

        if balance > 1 and key < root.left.key:
            return self.right_rotate(root)

        if balance < -1 and key > root.right.key:
            return self.left_rotate(root)

        if balance > 1 and key > root.left.key:
            root.left = self.left_rotate(root.left)
            return self.right_rotate(root)

        if balance < -1 and key < root.right.key:
            root.right = self.right_rotate(root.right)
            return self.left_rotate(root)

        return root
```

10.py - C:/Users/amuda/Desktop/DSA25124/10.py (3.14.0)

File Edit Format Run Options Window Help

```
        return root

    def min_value_node(self, root):
        current = root
        while current.left is not None:
            current = current.left
        return current

    def delete(self, root, key):
        if root is None:
            return root

        if key < root.key:
            root.left = self.delete(root.left, key)
        elif key > root.key:
            root.right = self.delete(root.right, key)
        else:
            if root.left is None:
                return root.right
            elif root.right is None:
                return root.left

            temp = self.min_value_node(root.right)
            root.key = temp.key
            root.right = self.delete(root.right, temp.key)

        if root is None:
            return root

        root.height = 1 + max(self.height(root.left),
                              self.height(root.right))

        balance = self.get_balance(root)

        if balance > 1 and self.get_balance(root.left) >= 0:
            return self.right_rotate(root)

        if balance > 1 and self.get_balance(root.left) < 0:
            root.left = self.left_rotate(root.left)
            return self.right_rotate(root)

        if balance < -1 and self.get_balance(root.right) <= 0:
            return self.left_rotate(root)
```

10.py - C:/Users/amuda/Desktop/DSA25124/10.py (3.14.0)

File Edit Format Run Options Window Help

```
        if balance < -1 and self.get_balance(root.right) > 0:
            root.right = self.right_rotate(root.right)
            return self.left_rotate(root)

        return root

    def search(self, root, key):
        if root is None or root.key == key:
            return root

        if key < root.key:
            return self.search(root.left, key)

        return self.search(root.right, key)

    def inorder(self, root):
        if root:
            self.inorder(root.left)
            print(root.key, end=" ")
            self.inorder(root.right)

    def preorder(self, root):
        if root:
            print(root.key, end=" ")
            self.preorder(root.left)
            self.preorder(root.right)

    def postorder(self, root):
        if root:
            self.postorder(root.left)
            self.postorder(root.right)
            print(root.key, end=" ")

    def find_min(self, root):
        if root is None:
            return None
        return self.min_value_node(root).key

    def find_max(self, root):
        if root is None:
            return None
        while root.right is not None:
            root = root.right
        return root.key
```

10.py - C:/Users/amuda/Desktop/DSA25124/10.py (3.14.0)

File Edit Format Run Options Window Help

```
tree = AVLTree()
root = None

while True:
    print("\n\n--- AVL TREE OPERATIONS ---")
    print("1. Insert")
    print("2. Delete")
    print("3. Search")
    print("4. Inorder Traversal")
    print("5. Preorder Traversal")
    print("6. Postorder Traversal")
    print("7. Find Minimum")
    print("8. Find Maximum")
    print("9. Display Height")
    print("10. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        key = int(input("Enter value to insert: "))
        root = tree.insert(root, key)
        print("Node inserted successfully")

    elif choice == 2:
        key = int(input("Enter value to delete: "))
        if tree.search(root, key):
            root = tree.delete(root, key)
            print("Node deleted successfully")
        else:
            print("Element not found")

    elif choice == 3:
        key = int(input("Enter value to search: "))
        if tree.search(root, key):
            print("Element found")
        else:
            print("Element not found")

    elif choice == 4:
        print("Inorder Traversal:", end=" ")
        tree.inorder(root)

    elif choice == 5:
        print("Preorder Traversal:", end=" ")
        tree.preorder(root)
```

```
    elif choice == 6:
        print("Postorder Traversal:", end=" ")
        tree.postorder(root)

    elif choice == 7:
        print("Minimum value:", tree.find_min(root))

    elif choice == 8:
        print("Maximum value:", tree.find_max(root))

    elif choice == 9:
        print("Tree Height:", tree.height(root))

    elif choice == 10:
        print("Exiting AVL Tree Program")
        break

    else:
        print("Invalid choice")
```

OUTPUT:

```
================ RESTART: C:/Users/amuda/Desktop/DSA25124/10.
Name: A.Likitha
Roll number:25124
--------------------


--- AVL TREE OPERATIONS ---
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Find Minimum
8. Find Maximum
9. Display Height
10. Exit
Enter your choice: 1
Enter value to insert: 10
Node inserted successfully


--- AVL TREE OPERATIONS ---
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Find Minimum
8. Find Maximum
9. Display Height
10. Exit
Enter your choice: 1
Enter value to insert: 15
Node inserted successfully


--- AVL TREE OPERATIONS ---
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
```

```
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Find Minimum
8. Find Maximum
9. Display Height
10. Exit
Enter your choice: 1
Enter value to insert: 20
Node inserted successfully


--- AVL TREE OPERATIONS ---
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Find Minimum
8. Find Maximum
9. Display Height
10. Exit
Enter your choice: 1
Enter value to insert: 25
Node inserted successfully


--- AVL TREE OPERATIONS ---
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Find Minimum
8. Find Maximum
9. Display Height
10. Exit
Enter your choice: 1
Enter value to insert: 30
Node inserted successfully


--- AVL TREE OPERATIONS ---
1. Insert
2. Delete
```

```
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Find Minimum
8. Find Maximum
9. Display Height
10. Exit
Enter your choice: 2
Enter value to delete: 25
Node deleted successfully


--- AVL TREE OPERATIONS ---
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Find Minimum
8. Find Maximum
9. Display Height
10. Exit
Enter your choice: 3
Enter value to search: 10
Element found


--- AVL TREE OPERATIONS ---
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Find Minimum
8. Find Maximum
9. Display Height
10. Exit
Enter your choice: 4
Inorder Traversal: 10 15 20 30
```

```
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Find Minimum
8. Find Maximum
9. Display Height
10. Exit
Enter your choice: 7
Minimum value: 10


--- AVL TREE OPERATIONS ---
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Find Minimum
8. Find Maximum
9. Display Height
10. Exit
Enter your choice: 8
Maximum value: 30


--- AVL TREE OPERATIONS ---
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Find Minimum
8. Find Maximum
9. Display Height
10. Exit
Enter your choice: 9
Tree Height: 3


--- AVL TREE OPERATIONS ---
1. Insert
2. Delete
3. Search
```

```
--- AVL TREE OPERATIONS ---
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Find Minimum
8. Find Maximum
9. Display Height
10. Exit
Enter your choice: 10
Exiting AVL Tree Program
```

ERROR TABLE:

S.NO	ERROR	RECTIFICATION
1.	Attribute Error	Check whether the node attributes are correctly initialized.
2.	Recursion Error	Ensure proper base conditions in recursive functions.
3.	Type Error	Enter valid integer values for tree operations.
4.	Node not found	Verify that the entered key exists before deletion or searching.4

EXPLANATION:

- A Node class is created to store the key, left child, right child, and height of each node.
- An AVLTree class is created to perform all AVL Tree operations.
- The height() function calculates the height of a node.
- The get_balance() function calculates the balance factor of a node.
- The right_rotate() function performs right rotation to balance the tree.
- The left_rotate() function performs left rotation to balance the tree.
- The insert() function inserts a new node following Binary Search Tree rules and balances the tree if required.
- The min_value_node() function finds the smallest value in the tree by moving towards the leftmost node.
- The delete() function removes a node and performs rotations if required to maintain balance.
- The search() function searches for a given element in the AVL Tree.
- The inorder() function displays elements in ascending order.

- The preorder() function displays the root, left subtree, and right subtree.
- The postorder() function displays the left subtree, right subtree, and root.
- The find_min() function returns the minimum value in the tree.
- The find_max() function returns the maximum value in the tree.
- A menu-driven program allows the user to select different AVL Tree operations.
- The while loop repeatedly displays the menu until the user selects the Exit option.
- The program maintains the AVL Tree balance using four rotations: LL, RR, LR, and RL.

WEEK-11

PROGRAM-11:

AIM: Write a python program to implement Hashing (Separate chaining, Linear probing, and Quadratic probing).

CODE:

11.py - C:/Users/amuda/Desktop/DSA25124/11.py (3.14.0)

File Edit Format Run Options Window Help

```
print("Name: A.Likitha")
print("Roll number:25124")
print("--------------------")
# Hashing using Separate Chaining, Linear Probing and Quadratic Probing

SIZE = 10


# Separate Chaining
class SeparateChaining:
    def __init__(self):
        self.table = [[] for _ in range(SIZE)]

    def insert(self, key):
        index = key % SIZE
        if key not in self.table[index]:
            self.table[index].append(key)

    def search(self, key):
        index = key % SIZE
        return key in self.table[index]

    def display(self):
        for i in range(SIZE):
            print(i, ":", self.table[i])


# Linear Probing
class LinearProbing:
    def __init__(self):
        self.table = [None] * SIZE

    def insert(self, key):
        index = key % SIZE

        for i in range(SIZE):
            new_index = (index + i) % SIZE

            if self.table[new_index] is None:
                self.table[new_index] = key
                return

        print("Hash table is full")

    def search(self, key):
        index = key % SIZE
```

11.py - C:/Users/amuda/Desktop/DSA25124/11.py (3.14.0)

File Edit Format Run Options Window Help

```
        index = key % SIZE

        for i in range(SIZE):
            new_index = (index + i) % SIZE

            if self.table[new_index] is None:
                return False

            if self.table[new_index] == key:
                return True

        return False

    def display(self):
        for i in range(SIZE):
            print(i, ":", self.table[i])


# Quadratic Probing
class QuadraticProbing:
    def __init__(self):
        self.table = [None] * SIZE

    def insert(self, key):
        index = key % SIZE

        for i in range(SIZE):
            new_index = (index + i * i) % SIZE

            if self.table[new_index] is None:
                self.table[new_index] = key
                return

        print("Hash table is full")

    def search(self, key):
        index = key % SIZE

        for i in range(SIZE):
            new_index = (index + i * i) % SIZE

            if self.table[new_index] is None:
                return False

            if self.table[new_index] == key:
                return True
```

11.py - C:/Users/amuda/Desktop/DSA25124/11.py (3.14.0)

File Edit Format Run Options Window Help

```
                return True

        return False

    def display(self):
        for i in range(SIZE):
            print(i, ":", self.table[i])


# Main Program
while True:
    print("\n--- HASHING OPERATIONS ---")
    print("1. Separate Chaining")
    print("2. Linear Probing")
    print("3. Quadratic Probing")
    print("4. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 4:
        print("Exiting Program")
        break

    if choice not in [1, 2, 3]:
        print("Invalid choice")
        continue

    if choice == 1:
        h = SeparateChaining()
    elif choice == 2:
        h = LinearProbing()
    else:
        h = QuadraticProbing()

    while True:
        print("\n1. Insert")
        print("2. Search")
        print("3. Display")
        print("4. Back to Main Menu")

        op = int(input("Enter operation: "))

        if op == 1:
            key = int(input("Enter key: "))
            h.insert(key)
```

```
        elif op == 2:
            key = int(input("Enter key to search: "))

            if h.search(key):
                print("Key found")
            else:
                print("Key not found")

        elif op == 3:
            h.display()

        elif op == 4:
            break

        else:
            print("Invalid operation")
```

OUTPUT:

```
Name: A.Likitha
Roll number:25124
--------------------

--- HASHING OPERATIONS ---
1. Separate Chaining
2. Linear Probing
3. Quadratic Probing
4. Exit
Enter your choice: 1

1. Insert
2. Search
3. Display
4. Back to Main Menu
Enter operation: 1
Enter key: 24

1. Insert
2. Search
3. Display
4. Back to Main Menu
Enter operation: 1
Enter key: 12

1. Insert
2. Search
3. Display
4. Back to Main Menu
Enter operation: 3
```

```
================ RESTART: C:/Users/amuda/Desktop/DSA25124/11.py =

--- HASHING OPERATIONS ---
1. Separate Chaining
2. Linear Probing
3. Quadratic Probing
4. Exit
Enter your choice: 1

1. Insert
2. Search
3. Display
4. Back to Main Menu
Enter operation: 1
Enter key: 24

1. Insert
2. Search
3. Display
4. Back to Main Menu
Enter operation: 2
Enter key to search: 24
Key found
```

ERROR TABLE:

S.NO	ERROR	RECTIFICATION
1.	Index Error	Use modulo(%) to keep the index within the hash table size.
2.	Value Error	Enter valid integer values instead of characters.
3.	Hash Collision	Use separate chaining, linear probing, or quadratic probing to resolve collisions.
4.	Invalid Choice	Enter a valid option from the menu.

EXPLANATION:

- A hash table of size 10 is created to store the keys.
- The hash function key % SIZE calculates the index for each key.
- Three hashing techniques are implemented: Separate Chaining, Linear Probing, and Quadratic Probing.
- Separate Chaining handles collisions by storing multiple keys in a list at the same index.

- Linear Probing resolves collisions by checking the next available position sequentially.
- Quadratic Probing resolves collisions by checking positions using the square of the probe number.
- The insert() function inserts keys into the hash table.
- The search() function checks whether a particular key is present.
- The display() function displays the contents of the hash table.
- A menu-driven program allows the user to select a hashing technique.
- The user can perform insertion, searching, and display operations.
- The Back to Main Menu option allows the user to select another hashing technique.
- The Exit option terminates the program.